SkyBase Intelligence Platform — Technical Architecture

Version: 1.0.0
Date: 2025
Authors: SkyBase Engineering
Classification: Internal Technical Reference

Table of Contents

1. System Overview
2. Directory Structure
3. Database Schema
4. API Endpoints
5. The Seven Agents
6. Agent Orchestration Layer
7. PDF Report Structure
8. Frontend (Next.js)
9. Payment Flow
10. Environment Variables
11. Deployment Architecture
12. Local Development Setup
13. Development Timeline Estimate

1. System Overview

1.1 Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                                CLIENT BROWSER                                 │
│  ┌──────────────────────────────────────────────────────────────────────┐    │
│  │  Next.js 14 (Vercel)                                                 │    │
│  │  / (landing) → /analyze → Stripe Checkout → /status/[id] → /report  │    │
│  └──────────────────────────────────────────────────────────────────────┘    │
└──────────────────────────────────────┬──────────────────────────────────────┘
                                       │ HTTPS
                                       ▼
                              ┌──────────────────┐
                              │   FastAPI Backend    │
```

```
                        |   (Railway / Docker) |
                        |                      |
                        |  /api/v1/analyses    |
                        |  /api/v1/webhook     |
                        |  /health             |
                        └──────┬──────────────┘
                               |         |
                    ┌──────────▼──┐   ┌──▼──────────────┐
                    |  PostgreSQL DB  |   |   Redis Broker   |
                    |  (Railway PG)   |   |  (Railway Redis) |
                    └─────────────────┘   └────────┬────────┘
                                                   |
                                  |
                 ┌────────────────────────────────▼────────────────────────┐
                 |                Celery Worker Pool                       |
                 |                                                         |
                 |  ┌──────────┐  ┌──────────┐  ┌──────────┐              |
                 |  | Airspace |  |  Zoning  |  |  Power   |              |
                 |  |  Agent   |  |  Agent   |  |  Agent   |              |
                 |  └──────────┘  └──────────┘  └──────────┘              |
                 |  ┌──────────┐  ┌──────────┐  ┌──────────┐              |
                 |  |Structural|  |Regulatory|  |   Cost   |              |
                 |  |  Agent   |  |  Agent   |  |  Agent   |              |
                 |  └──────────┘  └──────────┘  └──────────┘              |
                 |                  ┌──────────┐                           |
                 |                  |  Noise   |                           |
                 |                  |  Agent   |                           |
                 |                  └──────────┘                           |
                 |                                                         |
                 |  ┌──────────────────────────────────────┐               |
                 |  |    Aggregation + PDF Generation Task  |               |
                 |  |    (triggered by Celery chord)        |               |
                 |  └──────────────────────────────────────┘               |
                 └─────────────────────────────────────────────────────────┘
                                  |
                 ┌────────────────┼──────────────────┐
                 |                |                  |
     ┌──────────▼──┐   ┌──────────▼──┐   ┌──────────▼──┐
     |  AWS S3 /   |   |  SendGrid   |   |    Stripe   |
     |  CF R2      |   |  (Email)    |   |  (Payments) |
     └─────────────┘   └─────────────┘   └─────────────┘
```

1.2 Data Flow

```
User Action                    Backend                        Async Workers
──────────                     ───────                        ─────────────

1. Enter address          →   POST /api/v1/analyses/analyze
                              - Validate address
                              - Geocode (Nominatim → Google fallback)
                              - Create analyses row (status=pending)
                              - Create Stripe Checkout Session ($499)
                          ←   Return {checkout_url, analysis_id}

2. Stripe payment         →   Stripe Checkout (hosted page)
   completes                  Stripe sends POST to /webhook/stripe
```

```
                                - Verify webhook signature
                                - Mark analysis payment_status=paid
                                - Dispatch Celery group (7 agents)  ──────────→  agent tas
                            ←   Return 200 OK to Stripe

  3. Browser polls          →   GET /api/v1/analyses/{id}
     /status/[id]           ←   Return {status, agent_results[]}
     every 5s                    (Frontend shows progress bars)

  4. All agents complete    →   (Celery chord callback)
                                - Aggregate scores → overall score
                                - Render HTML → PDF (WeasyPrint)
                                - Upload PDF to S3
                                - Send email via SendGrid
                                - Update analyses.status = complete
                                - Update analyses.report_url

  5. User downloads         →   GET /api/v1/analyses/{id}/report
     report                 ←   302 redirect to signed S3 URL (15 min TTL)
```

1.3 Key Design Decisions

Why FastAPI over Django/Flask?
FastAPI gives us async-native request handling, automatic OpenAPI docs, and first-class Pydantic validation. For an API-only backend serving a Next.js frontend, the Django ORM and admin panel provide no value. FastAPI's dependency injection system makes auth, database sessions, and shared services trivially testable.

Why Celery + Redis over AWS SQS + Lambda?
The 30-minute analysis window with 7 parallel agents benefits from a persistent worker process rather than cold-starting Lambda. Celery's `group` and `chord` primitives express the parallel-then-aggregate pattern cleanly. Redis as both broker and result backend means one fewer managed service than SQS + ElastiCache. Upstash Redis gives us a free tier and serverless pricing that scales to zero.

Why WeasyPrint over Puppeteer/headless Chrome?
WeasyPrint produces byte-for-byte reproducible PDFs from HTML/CSS, with no browser binary to manage in Docker. The PDF is generated server-side with no JavaScript runtime, making it deterministic and fast (2–5 seconds for a 20-page report). ReportLab is kept as a fallback for environments where WeasyPrint's Cairo dependency causes issues.

Why PostgreSQL JSONB for agent raw_data?
Each agent returns a structurally different payload. Rather than 7 separate normalized tables with complex joins, JSONB lets each agent store its full response document while PostgreSQL still supports indexed queries on specific fields. The `score`, `summary`, and `warnings` fields are extracted as typed columns for the reports and APIs.

Why Railway over AWS ECS for the backend?
Railway deploys a Docker container directly from a GitHub repo with automatic deploys, managed TLS, and built-in PostgreSQL/Redis add-ons. For a 0-to-1 product, Railway eliminates

80% of the DevOps surface area. Migration to ECS is straightforward because the backend is containerized from day one.

Monorepo vs polyrepo:
Monorepo with a single GitHub repository. `backend/` and `frontend/` are separate deployable units (Railway and Vercel respectively). Shared `.env.example` and `docker-compose.yml` at the root make local development a single `docker compose up` command.

2. Directory Structure

```
skybase/
├── .github/
│   └── workflows/
│       ├── ci.yml                  # Test + lint on every PR
│       └── deploy.yml              # Deploy to Railway/Vercel on main merge
├── backend/
│   ├── app/
│   │   ├── __init__.py
│   │   ├── main.py                 # FastAPI app factory
│   │   ├── config.py               # Settings via pydantic-settings
│   │   ├── database.py             # SQLAlchemy engine + session factory
│   │   ├── celery_app.py           # Celery app instance
│   │   |
│   │   ├── api/
│   │   │   ├── __init__.py
│   │   │   ├── deps.py             # Shared FastAPI dependencies (DB session, auth
│   │   │   ├── router.py           # Root API router (v1 prefix)
│   │   │   └── v1/
│   │   │       ├── __init__.py
│   │   │       ├── analyses.py     # /api/v1/analyses routes
│   │   │       └── health.py       # /health route
│   │   |
│   │   ├── agents/
│   │   │   ├── __init__.py
│   │   │   ├── base.py             # BaseAgent abstract class
│   │   │   ├── agent_airspace.py    # FAA DOF + airspace classification
│   │   │   ├── agent_zoning.py      # State GIS + OSM land use
│   │   │   ├── agent_power.py       # EIA API + NREL + OSM substations
│   │   │   ├── agent_structural.py  # FAA EB 105A rules engine
│   │   │   ├── agent_regulatory.py   # Static regulatory ruleset DB
│   │   │   ├── agent_cost.py        # Internal cost model
│   │   │   └── agent_noise.py       # NOAA + FEMA + EPA EJScreen
│   │   |
│   │   ├── models/
│   │   │   ├── __init__.py
│   │   │   ├── base.py             # Declarative base
│   │   │   ├── analysis.py         # Analysis SQLAlchemy model
│   │   │   ├── agent_result.py      # AgentResult SQLAlchemy model
│   │   │   └── user.py             # User SQLAlchemy model
│   │   |
│   │   ├── schemas/
│   │   │   ├── __init__.py
│   │   │   ├── analysis.py         # Pydantic request/response schemas
```

```
│   │   │   ├── agent_result.py      # AgentResult schemas
│   │   │   └── user.py             # User schemas
│   │   │
│   │   ├── services/
│   │   │   ├── __init__.py
│   │   │   ├── geocoding.py         # Nominatim + Google Maps geocoder
│   │   │   ├── stripe_service.py    # Stripe Checkout + webhook logic
│   │   │   ├── pdf_service.py       # WeasyPrint + ReportLab PDF generation
│   │   │   ├── storage_service.py   # S3 / Cloudflare R2 upload + signed URLs
│   │   │   └── email_service.py     # SendGrid email delivery
│   │   │
│   │   ├── tasks/
│   │   │   ├── __init__.py
│   │   │   ├── orchestrator.py      # Celery group + chord dispatch
│   │   │   ├── task_airspace.py     # Celery task wrapper for airspace agent
│   │   │   ├── task_zoning.py
│   │   │   ├── task_power.py
│   │   │   ├── task_structural.py
│   │   │   ├── task_regulatory.py
│   │   │   ├── task_cost.py
│   │   │   ├── task_noise.py
│   │   │   └── task_aggregate.py    # Chord callback: aggregate + generate PDF
│   │   │
│   │   └── utils/
│   │       ├── __init__.py
│   │       ├── geo.py              # Haversine, bounding box helpers
│   │       ├── retry.py            # Exponential backoff decorator
│   │       └── logging.py          # Structured JSON logging setup
│   │
│   ├── migrations/
│   │   ├── env.py                  # Alembic env (auto-detects models)
│   │   ├── script.py.mako
│   │   └── versions/
│   │       └── 0001_initial.py     # Initial schema migration
│   │
│   ├── tests/
│   │   ├── conftest.py             # Pytest fixtures (test DB, test client)
│   │   ├── test_api/
│   │   │   ├── test_analyses.py
│   │   │   └── test_health.py
│   │   ├── test_agents/
│   │   │   ├── test_airspace.py
│   │   │   ├── test_zoning.py
│   │   │   └── test_cost.py
│   │   └── test_services/
│   │       ├── test_geocoding.py
│   │       └── test_pdf.py
│   │
│   ├── alembic.ini
│   ├── Dockerfile
│   ├── Dockerfile.worker            # Celery worker image (same code, different CI
│   ├── pyproject.toml               # Project metadata + tool config
│   └── requirements.txt
│
├── frontend/
│   ├── src/
```

```
│   │   ├── app/
│   │   │   ├── layout.tsx           # Root layout (fonts, metadata, providers)
│   │   │   ├── page.tsx             # / — landing page
│   │   │   ├── analyze/
│   │   │   │   └── page.tsx         # /analyze — address form + payment
│   │   │   ├── status/
│   │   │   │   └── [id]/
│   │   │   │       └── page.tsx     # /status/[id] — real-time progress
│   │   │   ├── report/
│   │   │   │   └── [id]/
│   │   │   │       └── page.tsx     # /report/[id] — PDF viewer + download
│   │   │   └── dashboard/
│   │   │       └── page.tsx         # /dashboard — past analyses (future)
│   │   │
│   │   ├── components/
│   │   │   ├── ui/                  # shadcn/ui base components
│   │   │   │   ├── button.tsx
│   │   │   │   ├── card.tsx
│   │   │   │   ├── badge.tsx
│   │   │   │   └── progress.tsx
│   │   │   ├── AddressInput.tsx      # Google Places autocomplete
│   │   │   ├── AnalysisProgress.tsx  # 7-agent progress bars
│   │   │   ├── ScoreGauge.tsx        # Circular readiness score
│   │   │   ├── ReportViewer.tsx      # PDF iframe + download
│   │   │   ├── PricingCard.tsx       # Pricing tiers
│   │   │   ├── Header.tsx
│   │   │   └── Footer.tsx
│   │   │
│   │   └── lib/
│   │       ├── api.ts               # Typed API client (fetch wrapper)
│   │       ├── stripe.ts            # Stripe.js loader
│   │       └── types.ts             # Shared TypeScript types
│   │
│   ├── public/
│   │   ├── logo.svg
│   │   └── sample-report-thumbnail.png
│   │
│   ├── .env.local.example
│   ├── next.config.ts
│   ├── tailwind.config.ts
│   ├── tsconfig.json
│   └── package.json
│
├── docker-compose.yml
├── .env.example
├── .gitignore
└── README.md
```

3. Database Schema

3.1 Base Model

```
# backend/app/models/base.py
import uuid
from datetime import datetime

from sqlalchemy import DateTime, func
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column


class Base(DeclarativeBase):
    pass


class TimestampMixin:
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now(),
        nullable=False,
    )
    updated_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now(),
        onupdate=func.now(),
        nullable=False,
    )
```

3.2 Analysis Model

```
# backend/app/models/analysis.py
import uuid
from datetime import datetime
from typing import Optional

from sqlalchemy import DateTime, Enum, Float, ForeignKey, String, Text
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.models.base import Base, TimestampMixin
import enum


class AnalysisStatus(str, enum.Enum):
    PENDING = "pending"
    PROCESSING = "processing"
    COMPLETE = "complete"
    FAILED = "failed"


class PaymentStatus(str, enum.Enum):
    UNPAID = "unpaid"
```

```
    PAID = "paid"
    REFUNDED = "refunded"


class Analysis(Base, TimestampMixin):
    __tablename__ = "analyses"

    id: Mapped[uuid.UUID] = mapped_column(
        UUID(as_uuid=True),
        primary_key=True,
        default=uuid.uuid4,
    )
    address_input: Mapped[str] = mapped_column(
        String(500),
        nullable=False,
        comment="Raw address string as entered by the user",
    )
    address_formatted: Mapped[Optional[str]] = mapped_column(
        String(500),
        nullable=True,
        comment="Normalized address returned by geocoder",
    )
    latitude: Mapped[Optional[float]] = mapped_column(
        Float,
        nullable=True,
    )
    longitude: Mapped[Optional[float]] = mapped_column(
        Float,
        nullable=True,
    )
    status: Mapped[AnalysisStatus] = mapped_column(
        Enum(AnalysisStatus, name="analysis_status_enum"),
        default=AnalysisStatus.PENDING,
        nullable=False,
        index=True,
    )
    payment_intent_id: Mapped[Optional[str]] = mapped_column(
        String(255),
        nullable=True,
        unique=True,
        comment="Stripe PaymentIntent or CheckoutSession ID",
    )
    payment_status: Mapped[PaymentStatus] = mapped_column(
        Enum(PaymentStatus, name="payment_status_enum"),
        default=PaymentStatus.UNPAID,
        nullable=False,
    )
    report_url: Mapped[Optional[str]] = mapped_column(
        String(2048),
        nullable=True,
        comment="S3 or R2 object key for the generated PDF",
    )
    user_email: Mapped[str] = mapped_column(
        String(320),
        nullable=False,
        index=True,
```

```
        comment="Email address for report delivery",
    )
    completed_at: Mapped[Optional[datetime]] = mapped_column(
        DateTime(timezone=True),
        nullable=True,
    )
    celery_task_id: Mapped[Optional[str]] = mapped_column(
        String(255),
        nullable=True,
        comment="Celery chord task ID for status tracking",
    )

    # Relationships
    agent_results: Mapped[list["AgentResult"]] = relationship(
        "AgentResult",
        back_populates="analysis",
        cascade="all, delete-orphan",
        order_by="AgentResult.agent_name",
    )

    @property
    def overall_score(self) -> Optional[int]:
        """Weighted average of all completed agent scores."""
        completed = [r for r in self.agent_results if r.score is not None]
        if not completed:
            return None
        # Weights: airspace 20%, zoning 20%, power 15%, structural 15%,
        #          regulatory 15%, cost 10%, noise 5%
        weights = {
            "airspace": 0.20,
            "zoning": 0.20,
            "power": 0.15,
            "structural": 0.15,
            "regulatory": 0.15,
            "cost": 0.10,
            "noise": 0.05,
        }
        total_weight = sum(weights.get(r.agent_name, 0.0) for r in completed)
        if total_weight == 0:
            return None
        weighted_sum = sum(
            (r.score or 0) * weights.get(r.agent_name, 0.0) for r in completed
        )
        return round(weighted_sum / total_weight)

    def __repr__(self) -> str:
        return f"<Analysis id={self.id} address={self.address_input!r} status={self.s
```

3.3 AgentResult Model

```
# backend/app/models/agent_result.py
import uuid
from datetime import datetime
from typing import Any, Optional
```

```
from sqlalchemy import DateTime, Enum, ForeignKey, Integer, String, Text
from sqlalchemy.dialects.postgresql import JSONB, UUID
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.models.base import Base
import enum


class AgentStatus(str, enum.Enum):
    PENDING = "pending"
    RUNNING = "running"
    COMPLETE = "complete"
    FAILED = "failed"


VALID_AGENT_NAMES = frozenset(
    ["airspace", "zoning", "power", "structural", "regulatory", "cost", "noise"]
)


class AgentResult(Base):
    __tablename__ = "agent_results"

    id: Mapped[uuid.UUID] = mapped_column(
        UUID(as_uuid=True),
        primary_key=True,
        default=uuid.uuid4,
    )
    analysis_id: Mapped[uuid.UUID] = mapped_column(
        UUID(as_uuid=True),
        ForeignKey("analyses.id", ondelete="CASCADE"),
        nullable=False,
        index=True,
    )
    agent_name: Mapped[str] = mapped_column(
        String(50),
        nullable=False,
        comment="One of: airspace, zoning, power, structural, regulatory, cost, noi
    )
    status: Mapped[AgentStatus] = mapped_column(
        Enum(AgentStatus, name="agent_status_enum"),
        default=AgentStatus.PENDING,
        nullable=False,
    )
    raw_data: Mapped[Optional[dict[str, Any]]] = mapped_column(
        JSONB,
        nullable=True,
        comment="Full API response payload from the data source",
    )
    score: Mapped[Optional[int]] = mapped_column(
        Integer,
        nullable=True,
        comment="Readiness score 0-100 for this dimension",
    )
    summary: Mapped[Optional[str]] = mapped_column(
        Text,
```

```
        nullable=True,
        comment="Human-readable 2-3 sentence summary for the report",
    )
    warnings: Mapped[Optional[list[Any]]] = mapped_column(
        JSONB,
        nullable=True,
        default=list,
        comment="Array of warning strings surfaced in the report",
    )
    started_at: Mapped[Optional[datetime]] = mapped_column(
        DateTime(timezone=True),
        nullable=True,
    )
    completed_at: Mapped[Optional[datetime]] = mapped_column(
        DateTime(timezone=True),
        nullable=True,
    )
    error_message: Mapped[Optional[str]] = mapped_column(
        Text,
        nullable=True,
        comment="Exception message if status=failed",
    )

    # Relationship
    analysis: Mapped["Analysis"] = relationship(
        "Analysis",
        back_populates="agent_results",
    )

    def __repr__(self) -> str:
        return (
            f"<AgentResult agent={self.agent_name} "
            f"analysis_id={self.analysis_id} score={self.score}>"
        )
```

3.4 User Model

```
# backend/app/models/user.py
import uuid
from datetime import datetime
from typing import Optional

from sqlalchemy import DateTime, Enum, String
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import Mapped, mapped_column

from app.models.base import Base, TimestampMixin
import enum


class UserTier(str, enum.Enum):
    FREE = "free"
    PRO = "pro"
    ENTERPRISE = "enterprise"
```

```
class User(Base, TimestampMixin):
    __tablename__ = "users"

    id: Mapped[uuid.UUID] = mapped_column(
        UUID(as_uuid=True),
        primary_key=True,
        default=uuid.uuid4,
    )
    email: Mapped[str] = mapped_column(
        String(320),
        nullable=False,
        unique=True,
        index=True,
    )
    tier: Mapped[UserTier] = mapped_column(
        Enum(UserTier, name="user_tier_enum"),
        default=UserTier.FREE,
        nullable=False,
    )
    stripe_customer_id: Mapped[Optional[str]] = mapped_column(
        String(255),
        nullable=True,
        unique=True,
        comment="Stripe Customer ID for recurring billing",
    )
    reports_remaining: Mapped[int] = mapped_column(
        default=0,
        nullable=False,
        comment="Pre-purchased report credits (bulk/subscription tiers)",
    )

    def __repr__(self) -> str:
        return f"<User email={self.email!r} tier={self.tier}>"
```

3.5 Alembic Migration — Initial Schema

```
# backend/migrations/versions/0001_initial.py
"""Initial schema

Revision ID: 0001
Create Date: 2025-01-01 00:00:00.000000
"""
from alembic import op
import sqlalchemy as sa
from sqlalchemy.dialects import postgresql

revision = "0001"
down_revision = None
branch_labels = None
depends_on = None


def upgrade() -> None:
    # Enums
```

```
    analysis_status = postgresql.ENUM(
        "pending", "processing", "complete", "failed",
        name="analysis_status_enum",
    )
    payment_status = postgresql.ENUM(
        "unpaid", "paid", "refunded",
        name="payment_status_enum",
    )
    agent_status = postgresql.ENUM(
        "pending", "running", "complete", "failed",
        name="agent_status_enum",
    )
    user_tier = postgresql.ENUM(
        "free", "pro", "enterprise",
        name="user_tier_enum",
    )

    analysis_status.create(op.get_bind(), checkfirst=True)
    payment_status.create(op.get_bind(), checkfirst=True)
    agent_status.create(op.get_bind(), checkfirst=True)
    user_tier.create(op.get_bind(), checkfirst=True)

    op.create_table(
        "users",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("email", sa.String(320), nullable=False, unique=True),
        sa.Column("tier", sa.Enum("free", "pro", "enterprise", name="user_tier_enu
        sa.Column("stripe_customer_id", sa.String(255), nullable=True, unique=True),
        sa.Column("reports_remaining", sa.Integer(), nullable=False, server_default=
        sa.Column("created_at", sa.DateTime(timezone=True), server_default=sa.func.n
        sa.Column("updated_at", sa.DateTime(timezone=True), server_default=sa.func.n
    )
    op.create_index("ix_users_email", "users", ["email"])

    op.create_table(
        "analyses",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("address_input", sa.String(500), nullable=False),
        sa.Column("address_formatted", sa.String(500), nullable=True),
        sa.Column("latitude", sa.Float(), nullable=True),
        sa.Column("longitude", sa.Float(), nullable=True),
        sa.Column("status", sa.Enum("pending", "processing", "complete", "failed",
        sa.Column("payment_intent_id", sa.String(255), nullable=True, unique=True),
        sa.Column("payment_status", sa.Enum("unpaid", "paid", "refunded", name="pay
        sa.Column("report_url", sa.String(2048), nullable=True),
        sa.Column("user_email", sa.String(320), nullable=False),
        sa.Column("completed_at", sa.DateTime(timezone=True), nullable=True),
        sa.Column("celery_task_id", sa.String(255), nullable=True),
        sa.Column("created_at", sa.DateTime(timezone=True), server_default=sa.func.n
        sa.Column("updated_at", sa.DateTime(timezone=True), server_default=sa.func.n
    )
    op.create_index("ix_analyses_status", "analyses", ["status"])
    op.create_index("ix_analyses_user_email", "analyses", ["user_email"])

    op.create_table(
        "agent_results",
```

```
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("analysis_id", postgresql.UUID(as_uuid=True), sa.ForeignKey("analy
        sa.Column("agent_name", sa.String(50), nullable=False),
        sa.Column("status", sa.Enum("pending", "running", "complete", "failed", nam
        sa.Column("raw_data", postgresql.JSONB(), nullable=True),
        sa.Column("score", sa.Integer(), nullable=True),
        sa.Column("summary", sa.Text(), nullable=True),
        sa.Column("warnings", postgresql.JSONB(), nullable=True),
        sa.Column("started_at", sa.DateTime(timezone=True), nullable=True),
        sa.Column("completed_at", sa.DateTime(timezone=True), nullable=True),
        sa.Column("error_message", sa.Text(), nullable=True),
    )
    op.create_index("ix_agent_results_analysis_id", "agent_results", ["analysis_id"]


def downgrade() -> None:
    op.drop_table("agent_results")
    op.drop_table("analyses")
    op.drop_table("users")
    for enum_name in [
        "analysis_status_enum", "payment_status_enum",
        "agent_status_enum", "user_tier_enum",
    ]:
        op.execute(f"DROP TYPE IF EXISTS {enum_name}")
```

4. API Endpoints (FastAPI Routers)

4.1 Pydantic Schemas

```
# backend/app/schemas/analysis.py
import uuid
from datetime import datetime
from typing import Any, Optional

from pydantic import BaseModel, EmailStr, Field


# ── Request schemas ──────────────────────────────────────────────────────────

class AnalyzeRequest(BaseModel):
    address: str = Field(
        ...,
        min_length=5,
        max_length=500,
        description="Full US property address",
        examples=["1234 NW 56th St, Miami, FL 33142"],
    )
    email: EmailStr = Field(
        ...,
        description="Email address for report delivery",
    )
    property_type: str = Field(
        default="unknown",
```

```
        description="One of: rooftop, ground, airport, unknown",
        pattern="^(rooftop|ground|airport|unknown)$",
    )
    build_year: Optional[int] = Field(
        default=None,
        ge=1800,
        le=2025,
        description="Year property was constructed (for structural agent)",
    )
    num_stories: Optional[int] = Field(
        default=None,
        ge=1,
        le=200,
        description="Number of stories (for structural agent)",
    )


# ── Response schemas ──────────────────────────────────────────────────────

class AgentResultResponse(BaseModel):
    id: uuid.UUID
    agent_name: str
    status: str
    score: Optional[int] = None
    summary: Optional[str] = None
    warnings: Optional[list[str]] = None
    started_at: Optional[datetime] = None
    completed_at: Optional[datetime] = None
    error_message: Optional[str] = None

    model_config = {"from_attributes": True}


class AnalysisResponse(BaseModel):
    id: uuid.UUID
    address_input: str
    address_formatted: Optional[str] = None
    latitude: Optional[float] = None
    longitude: Optional[float] = None
    status: str
    payment_status: str
    report_url: Optional[str] = None
    overall_score: Optional[int] = None
    created_at: datetime
    completed_at: Optional[datetime] = None
    agent_results: list[AgentResultResponse] = []

    model_config = {"from_attributes": True}


class CreateAnalysisResponse(BaseModel):
    analysis_id: uuid.UUID
    checkout_url: str
    message: str = "Redirect user to checkout_url to complete payment."
```

```
class ReportRedirectResponse(BaseModel):
    download_url: str
    expires_in_seconds: int = 900
```

4.2 App Factory

```
# backend/app/main.py
from contextlib import asynccontextmanager

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.middleware.trustedhost import TrustedHostMiddleware

from app.api.router import api_router
from app.config import settings
from app.database import engine
from app.models.base import Base
from app.utils.logging import configure_logging


@asynccontextmanager
async def lifespan(app: FastAPI):
    configure_logging()
    # Tables are managed by Alembic; this is a safety net for tests
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
    yield
    await engine.dispose()


def create_app() -> FastAPI:
    app = FastAPI(
        title="SkyBase Intelligence Platform API",
        description="Vertiport readiness analysis for US properties",
        version="1.0.0",
        docs_url="/docs" if settings.ENVIRONMENT != "production" else None,
        redoc_url="/redoc" if settings.ENVIRONMENT != "production" else None,
        lifespan=lifespan,
    )

    app.add_middleware(
        CORSMiddleware,
        allow_origins=settings.CORS_ORIGINS,
        allow_credentials=True,
        allow_methods=["GET", "POST", "PUT", "DELETE"],
        allow_headers=["*"],
    )

    if settings.ENVIRONMENT == "production":
        app.add_middleware(
            TrustedHostMiddleware,
            allowed_hosts=["api.skybaseintel.com"],
        )

    app.include_router(api_router)
```

```
    return app


app = create_app()
```

4.3 Configuration

```
# backend/app/config.py
from functools import lru_cache
from typing import Optional

from pydantic_settings import BaseSettings, SettingsConfigDict


class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        case_sensitive=False,
    )

    ENVIRONMENT: str = "development"

    # Database
    DATABASE_URL: str

    # Redis / Celery
    REDIS_URL: str = "redis://localhost:6379/0"

    # Stripe
    STRIPE_SECRET_KEY: str
    STRIPE_WEBHOOK_SECRET: str
    STRIPE_PRICE_ID: str  # price_xxx for the $499 product

    # Google Maps
    GOOGLE_MAPS_API_KEY: str

    # EIA API
    EIA_API_KEY: str

    # NREL
    NREL_API_KEY: str

    # AWS S3 / Cloudflare R2
    AWS_S3_BUCKET: str
    AWS_ACCESS_KEY_ID: str
    AWS_SECRET_ACCESS_KEY: str
    AWS_S3_ENDPOINT_URL: Optional[str] = None  # Set for Cloudflare R2
    AWS_REGION: str = "us-east-1"

    # SendGrid
    SENDGRID_API_KEY: str
    FROM_EMAIL: str = "reports@skybaseintel.com"

    # Frontend URL (for Stripe redirect)
```

```
    FRONTEND_URL: str = "https://skybaseintel.com"

    # CORS
    CORS_ORIGINS: list[str] = ["https://skybaseintel.com"]

    # Analysis price in cents
    ANALYSIS_PRICE_CENTS: int = 49900


@lru_cache
def get_settings() -> Settings:
    return Settings()


settings = get_settings()
```

4.4 Database Session

```
# backend/app/database.py
from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker, create_async_er

from app.config import settings

# Convert postgresql:// to postgresql+asyncpg://
_db_url = settings.DATABASE_URL.replace(
    "postgresql://", "postgresql+asyncpg://", 1
).replace(
    "postgres://", "postgresql+asyncpg://", 1
)

engine = create_async_engine(
    _db_url,
    pool_size=10,
    max_overflow=20,
    pool_pre_ping=True,
    echo=settings.ENVIRONMENT == "development",
)

AsyncSessionLocal = async_sessionmaker(
    bind=engine,
    class_=AsyncSession,
    expire_on_commit=False,
    autocommit=False,
    autoflush=False,
)
```

4.5 Dependencies

```
# backend/app/api/deps.py
from typing import AsyncGenerator

from fastapi import Depends
from sqlalchemy.ext.asyncio import AsyncSession
```

```
from app.database import AsyncSessionLocal


async def get_db() -> AsyncGenerator[AsyncSession, None]:
    async with AsyncSessionLocal() as session:
        try:
            yield session
            await session.commit()
        except Exception:
            await session.rollback()
            raise
        finally:
            await session.close()
```

4.6 Analyses Router

```
# backend/app/api/v1/analyses.py
import hashlib
import hmac
import uuid
from typing import Annotated

import stripe
from fastapi import APIRouter, BackgroundTasks, Depends, Header, HTTPException, Rec
from fastapi.responses import JSONResponse, RedirectResponse
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.orm import selectinload

from app.api.deps import get_db
from app.config import settings
from app.models.analysis import Analysis, AnalysisStatus, PaymentStatus
from app.models.agent_result import AgentResult, AgentStatus, VALID_AGENT_NAMES
from app.schemas.analysis import (
    AnalyzeRequest,
    AnalysisResponse,
    CreateAnalysisResponse,
    ReportRedirectResponse,
)
from app.services.geocoding import geocode_address
from app.services.storage_service import generate_signed_url
from app.services.stripe_service import create_checkout_session
from app.tasks.orchestrator import dispatch_analysis

router = APIRouter(prefix="/analyses", tags=["analyses"])

stripe.api_key = settings.STRIPE_SECRET_KEY


# ── POST /analyze ──────────────────────────────────────────────────────────

@router.post("/analyze", response_model=CreateAnalysisResponse, status_code=201)
async def create_analysis(
    payload: AnalyzeRequest,
```

```
    db: Annotated[AsyncSession, Depends(get_db)],
) -> CreateAnalysisResponse:
    """
    Create a new analysis record and return a Stripe Checkout URL.
    Payment must complete before agent tasks are dispatched.
    """
    # Geocode the address
    geo = await geocode_address(payload.address)
    if geo is None:
        raise HTTPException(
            status_code=422,
            detail="Unable to geocode address. Please enter a valid US address.",
        )

    # Create the analysis record
    analysis = Analysis(
        address_input=payload.address,
        address_formatted=geo["formatted_address"],
        latitude=geo["lat"],
        longitude=geo["lng"],
        user_email=payload.email,
        status=AnalysisStatus.PENDING,
        payment_status=PaymentStatus.UNPAID,
    )
    db.add(analysis)

    # Create placeholder AgentResult rows (pending) for UI progress display
    for agent_name in VALID_AGENT_NAMES:
        db.add(
            AgentResult(
                analysis_id=analysis.id,
                agent_name=agent_name,
                status=AgentStatus.PENDING,
            )
        )

    await db.flush()  # Get the analysis.id before commit

    # Create Stripe Checkout Session
    checkout_session = await create_checkout_session(
        analysis_id=str(analysis.id),
        email=payload.email,
        metadata={
            "analysis_id": str(analysis.id),
            "address": payload.address,
            "property_type": payload.property_type,
            "build_year": str(payload.build_year or ""),
            "num_stories": str(payload.num_stories or ""),
        },
    )

    analysis.payment_intent_id = checkout_session.id
    await db.commit()

    return CreateAnalysisResponse(
        analysis_id=analysis.id,
```

```
        checkout_url=checkout_session.url,
    )


# ── GET /{analysis_id} ──────────────────────────────────────────────────────

@router.get("/{analysis_id}", response_model=AnalysisResponse)
async def get_analysis(
    analysis_id: uuid.UUID,
    db: Annotated[AsyncSession, Depends(get_db)],
) -> AnalysisResponse:
    """Return current status and all agent results for an analysis."""
    result = await db.execute(
        select(Analysis)
        .options(selectinload(Analysis.agent_results))
        .where(Analysis.id == analysis_id)
    )
    analysis = result.scalar_one_or_none()
    if analysis is None:
        raise HTTPException(status_code=404, detail="Analysis not found")

    response = AnalysisResponse.model_validate(analysis)
    response.overall_score = analysis.overall_score
    return response


# ── GET /{analysis_id}/report ───────────────────────────────────────────────

@router.get("/{analysis_id}/report", response_model=ReportRedirectResponse)
async def get_report(
    analysis_id: uuid.UUID,
    db: Annotated[AsyncSession, Depends(get_db)],
) -> ReportRedirectResponse:
    """
    Generate a 15-minute signed S3 URL for the completed report PDF.
    Returns 402 if payment has not been confirmed.
    Returns 404 if analysis or report does not exist.
    Returns 425 if analysis is still in progress.
    """
    result = await db.execute(
        select(Analysis).where(Analysis.id == analysis_id)
    )
    analysis = result.scalar_one_or_none()
    if analysis is None:
        raise HTTPException(status_code=404, detail="Analysis not found")

    if analysis.payment_status != PaymentStatus.PAID:
        raise HTTPException(status_code=402, detail="Payment required")

    if analysis.status != AnalysisStatus.COMPLETE:
        raise HTTPException(
            status_code=425,
            detail=f"Analysis not yet complete (status: {analysis.status})",
        )

    if not analysis.report_url:
```

```
        raise HTTPException(
            status_code=404,
            detail="Report not yet available. Please try again in a moment.",
        )

    signed_url = generate_signed_url(analysis.report_url, expires_in=900)
    return ReportRedirectResponse(download_url=signed_url, expires_in_seconds=900)


# ── POST /webhook/stripe ──────────────────────────────────────────────────

@router.post("/webhook/stripe", status_code=200)
async def stripe_webhook(
    request: Request,
    db: Annotated[AsyncSession, Depends(get_db)],
    stripe_signature: Annotated[str | None, Header(alias="stripe-signature")] = None
) -> JSONResponse:
    """
    Stripe webhook handler. Verifies signature then processes events.
    Triggers Celery task group on checkout.session.completed.
    """
    body = await request.body()

    # Verify webhook signature
    try:
        event = stripe.Webhook.construct_event(
            payload=body,
            sig_header=stripe_signature,
            secret=settings.STRIPE_WEBHOOK_SECRET,
        )
    except stripe.error.SignatureVerificationError:
        raise HTTPException(status_code=400, detail="Invalid Stripe signature")
    except Exception as e:
        raise HTTPException(status_code=400, detail=f"Webhook error: {str(e)}")

    if event["type"] == "checkout.session.completed":
        session = event["data"]["object"]
        analysis_id_str = session.get("metadata", {}).get("analysis_id")

        if not analysis_id_str:
            return JSONResponse({"received": True, "warning": "No analysis_id in me

        try:
            analysis_id = uuid.UUID(analysis_id_str)
        except ValueError:
            return JSONResponse({"received": True, "warning": "Invalid analysis_id"}

        result = await db.execute(
            select(Analysis).where(Analysis.id == analysis_id)
        )
        analysis = result.scalar_one_or_none()
        if analysis is None:
            return JSONResponse({"received": True, "warning": "Analysis not found"})

        # Mark as paid and kick off agents
        analysis.payment_status = PaymentStatus.PAID
```

```
        analysis.status = AnalysisStatus.PROCESSING
        await db.flush()

        # Gather metadata passed through Stripe
        metadata = session.get("metadata", {})
        property_context = {
            "property_type": metadata.get("property_type", "unknown"),
            "build_year": int(metadata.get("build_year") or 0) or None,
            "num_stories": int(metadata.get("num_stories") or 0) or None,
        }

        # Dispatch Celery group (non-blocking)
        task = dispatch_analysis(
            analysis_id=str(analysis.id),
            lat=analysis.latitude,
            lon=analysis.longitude,
            address=analysis.address_formatted or analysis.address_input,
            property_context=property_context,
        )
        analysis.celery_task_id = task.id
        await db.commit()

    return JSONResponse({"received": True})
```

4.7 Health Router

```
# backend/app/api/v1/health.py
from fastapi import APIRouter, Depends
from sqlalchemy import text
from sqlalchemy.ext.asyncio import AsyncSession

from app.api.deps import get_db

router = APIRouter(tags=["health"])


@router.get("/health")
async def health_check(db: AsyncSession = Depends(get_db)) -> dict:
    """Returns service health including database connectivity."""
    try:
        await db.execute(text("SELECT 1"))
        db_status = "ok"
    except Exception as e:
        db_status = f"error: {str(e)}"

    return {
        "status": "ok" if db_status == "ok" else "degraded",
        "version": "1.0.0",
        "database": db_status,
    }
```

4.8 Root Router

```
# backend/app/api/router.py
from fastapi import APIRouter

from app.api.v1 import analyses, health

api_router = APIRouter()
api_router.include_router(health.router)
api_router.include_router(analyses.router, prefix="/api/v1")
```

5. The Seven Agents (Celery Tasks)

5.1 Base Agent

```
# backend/app/agents/base.py
import logging
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from datetime import datetime, timezone
from typing import Any, Optional

logger = logging.getLogger(__name__)


@dataclass
class AgentResult:
    score: int                      # 0-100
    summary: str                    # 2-3 sentence narrative
    warnings: list[str]             # Warning strings for report
    raw_data: dict[str, Any]        # Full data payload stored in JSONB
    recommendations: list[str] = field(default_factory=list)


class BaseAgent(ABC):
    """Abstract base class for all SkyBase analysis agents."""

    name: str = ""
    timeout_seconds: int = 300

    def __init__(self, lat: float, lon: float, address: str, context: dict[str, Any]
        self.lat = lat
        self.lon = lon
        self.address = address
        self.context = context
        self.logger = logging.getLogger(f"agents.{self.name}")

    @abstractmethod
    async def run(self) -> AgentResult:
        """Execute the agent analysis. Must be implemented by each subclass."""
        ...
```

```
    def clamp_score(self, score: float) -> int:
        return max(0, min(100, round(score)))
```

5.2 Agent 1 — Airspace Agent

```
# backend/app/agents/agent_airspace.py
"""
Airspace Agent — queries FAA data to classify airspace and identify constraints.

Data sources:
  - FAA NASR (National Airspace System Resources) subscription:
    https://www.faa.gov/air_traffic/flight_info/aeronav/aero_data/NASR_Subscription/
  - FAA Digital Obstacle File (DOF) bulk CSV:
    https://oeaaa.faa.gov/oeaaa/external/gisTools/gisAction.jsp?action=getAirspaceDa
  - OpenAIP API for airport proximity (free tier):
    https://api.core.openaip.net/api/airports
"""
import math
from typing import Any

import httpx

from app.agents.base import AgentResult, BaseAgent
from app.utils.geo import haversine_miles
from app.utils.retry import async_retry


# Airspace class → base score penalty
AIRSPACE_SCORE_MAP = {
    "B": 0,    # Class B — most restrictive (major metro airports)
    "C": 25,   # Class C — regional airports
    "D": 45,   # Class D — towered general aviation
    "E": 65,   # Class E — controlled, no tower required
    "G": 90,   # Class G — uncontrolled, most permissive
}


class AirspaceAgent(BaseAgent):
    name = "airspace"

    @async_retry(max_attempts=3, base_delay=2.0)
    async def _get_nearby_airports(self) -> list[dict]:
        """Query OpenAIP for airports within 25 miles."""
        url = "https://api.core.openaip.net/api/airports"
        params = {
            "pos": f"{self.lat},{self.lon}",
            "dist": 40000,  # meters — ~25 miles
            "limit": 20,
        }
        async with httpx.AsyncClient(timeout=30) as client:
            r = await client.get(url, params=params)
            r.raise_for_status()
            return r.json().get("items", [])

    async def run(self) -> AgentResult:
```

```
        airports = await self._get_nearby_airports()
        warnings = []
        raw_data: dict[str, Any] = {
            "query_lat": self.lat,
            "query_lon": self.lon,
            "airports_found": len(airports),
        }

        # Find nearest airport
        nearest_dist = float("inf")
        nearest_name = None
        nearest_class = "G"

        airport_details = []
        for ap in airports:
            ap_lat = ap.get("geometry", {}).get("coordinates", [0, 0])[1]
            ap_lon = ap.get("geometry", {}).get("coordinates", [0, 0])[0]
            dist = haversine_miles(self.lat, self.lon, ap_lat, ap_lon)
            airspace_cls = ap.get("type", {}).get("airac", {}).get("class", "G")
            airport_details.append({
                "name": ap.get("name"),
                "icao": ap.get("icaoCode"),
                "distance_miles": round(dist, 2),
                "airspace_class": airspace_cls,
            })
            if dist < nearest_dist:
                nearest_dist = dist
                nearest_name = ap.get("name")
                nearest_class = airspace_cls

        airport_details.sort(key=lambda x: x["distance_miles"])
        raw_data["airports"] = airport_details[:10]
        raw_data["nearest_airport"] = nearest_name
        raw_data["nearest_airport_dist_miles"] = round(nearest_dist, 2)
        raw_data["nearest_airspace_class"] = nearest_class

        # Score from airspace class
        base_score = AIRSPACE_SCORE_MAP.get(nearest_class, 65)

        # Proximity penalties
        proximity_penalty = 0
        if nearest_dist < 1:
            proximity_penalty = 40
            warnings.append(
                f"CRITICAL: Within 1 mile of {nearest_name}. "
                "FAA Part 77 obstruction evaluation mandatory. Approval highly unlike
            )
        elif nearest_dist < 3:
            proximity_penalty = 25
            warnings.append(
                f"Airport within 3 miles ({nearest_name}, {nearest_dist:.1f} mi). "
                "FAA Form 7480-1 Notice of Proposed Construction mandatory. "
                "FAA airspace coordination required."
            )
        elif nearest_dist < 5:
            proximity_penalty = 10
```

```
            warnings.append(
                f"Within 5 miles of Class {nearest_class} airspace ({nearest_name}). 
                "FAA coordination required. UAS Remote ID compliance mandatory."
            )

        if nearest_class in ("B", "C"):
            warnings.append(
                f"Class {nearest_class} airspace — FAA Part 77 evaluation required. "
                "Expect 60-90 day FAA review. Altitude restrictions likely."
            )

        score = self.clamp_score(base_score - proximity_penalty)

        summary = (
            f"The site is located {nearest_dist:.1f} miles from {nearest_name or 'th
            f"in Class {nearest_class} airspace. "
        )
        if score >= 70:
            summary += "Airspace classification is favorable for vertiport developme
        elif score >= 40:
            summary += "Airspace constraints require FAA coordination and may impose
        else:
            summary += "Significant airspace restrictions present. FAA approval is u

        return AgentResult(
            score=score,
            summary=summary,
            warnings=warnings,
            raw_data=raw_data,
            recommendations=[
                "File FAA Form 7480-1 at least 45 days before construction",
                "Obtain FAA airspace authorization via LAANC or DroneZone",
                f"Review FAA VFR sectional chart for Class {nearest_class} boundaries"
            ],
        )
```

5.3 Agent 2 — Zoning Agent

```
# backend/app/agents/agent_zoning.py
"""
Zoning Agent — determines land use classification and vertiport permitting pathway.

Data sources:
  - State GIS portals (REST API where available):
      FL: https://services1.arcgis.com/CY1LXxl9zlJeBuiP/arcgis/rest/services/
      TX: https://gis.dallascityhall.com/arcgis/rest/services/
      CA: https://gis.data.ca.gov/
      NY: https://data.ny.gov/api/geospatial/
      IL: https://gis.illinois.gov/
  - OpenStreetMap Overpass API for land use polygons (fallback):
      https://overpass-api.de/api/interpreter
"""
import httpx

from app.agents.base import AgentResult, BaseAgent
```

```
from app.utils.retry import async_retry


# Zoning type → (base_score, description, permit_path)
ZONING_PROFILES = {
    # High suitability
    "aviation": (95, "Aviation/Airport District", "Permitted by right"),
    "industrial_heavy": (85, "Heavy Industrial", "Permitted by right or minor CUP")
    "industrial": (80, "Industrial", "Conditional Use Permit likely required"),
    "industrial_light": (75, "Light Industrial", "Conditional Use Permit required")
    "commercial_heavy": (65, "Heavy Commercial", "Conditional Use Permit required'
    "commercial": (55, "Commercial", "CUP required; 3-9 month timeline"),
    "commercial_light": (45, "Light Commercial", "CUP required; 6-12 month timelir
    # Low suitability
    "mixed_use": (40, "Mixed Use", "CUP required; noise/design review likely"),
    "office": (35, "Office/Business Park", "CUP required; 6-18 month timeline"),
    "residential_multi": (15, "Multi-Family Residential", "Likely prohibited; varia
    "residential": (5, "Single-Family Residential", "Prohibited in most jurisdictic
    "historic": (20, "Historic District", "Historic preservation review required")
    "open_space": (10, "Open Space/Park", "Prohibited in most jurisdictions"),
    "unknown": (50, "Unknown/Unclassified", "Manual zoning review required"),
}

# OSM landuse/aeroway tags → our zoning profile
OSM_TAG_MAP = {
    "aerodrome": "aviation",
    "aeroway": "aviation",
    "industrial": "industrial",
    "commercial": "commercial",
    "retail": "commercial_light",
    "office": "office",
    "residential": "residential",
    "apartments": "residential_multi",
    "mixed_use": "mixed_use",
    "historic": "historic",
    "park": "open_space",
    "nature_reserve": "open_space",
}


class ZoningAgent(BaseAgent):
    name = "zoning"

    @async_retry(max_attempts=3, base_delay=2.0)
    async def _query_osm_landuse(self) -> dict:
        """Query OpenStreetMap Overpass for land use at coordinates."""
        # Bounding box: ~0.25 mile radius
        delta = 0.004
        bbox = f"{self.lat - delta},{self.lon - delta},{self.lat + delta},{self.lon +
        query = f"""
        [out:json][timeout:25];
        (
          way[landuse]({bbox});
          way[aeroway]({bbox});
          relation[landuse]({bbox});
        );
```

```
        out body;
        """
        async with httpx.AsyncClient(timeout=30) as client:
            r = await client.post(
                "https://overpass-api.de/api/interpreter",
                data={"data": query},
            )
            r.raise_for_status()
            return r.json()

    def _classify_osm(self, osm_data: dict) -> str:
        """Map OSM landuse tags to our zoning profile key."""
        elements = osm_data.get("elements", [])
        for el in elements:
            tags = el.get("tags", {})
            for tag_key in ["aeroway", "landuse", "building"]:
                tag_val = tags.get(tag_key, "")
                if tag_val in OSM_TAG_MAP:
                    return OSM_TAG_MAP[tag_val]
        return "unknown"

    async def run(self) -> AgentResult:
        warnings = []

        try:
            osm_data = await self._query_osm_landuse()
            zoning_key = self._classify_osm(osm_data)
            data_source = "OpenStreetMap"
        except Exception as e:
            self.logger.warning(f"OSM query failed: {e}")
            osm_data = {}
            zoning_key = "unknown"
            data_source = "unavailable"

        score_base, zoning_label, permit_path = ZONING_PROFILES.get(
            zoning_key, ZONING_PROFILES["unknown"]
        )

        raw_data = {
            "zoning_classification": zoning_key,
            "zoning_label": zoning_label,
            "permit_pathway": permit_path,
            "data_source": data_source,
            "lat": self.lat,
            "lon": self.lon,
        }

        # Apply warning-based penalties
        if zoning_key == "residential":
            warnings.append(
                "Residential zoning — vertiport use is prohibited in most jurisdicti
                "Rezoning or variance required; expect 12-24 month process."
            )
        elif zoning_key == "residential_multi":
            warnings.append(
                "Multi-family residential zoning — conditional use permit required.
```

```
                "Noise and safety review mandatory; 6-18 month timeline typical."
            )
        elif zoning_key == "historic":
            warnings.append(
                "Historic district overlay detected — Historic Preservation Commiss
                "Structural modifications may be restricted."
            )
        elif zoning_key == "commercial":
            warnings.append(
                "Commercial zoning requires Conditional Use Permit (CUP). "
                "Public hearing likely; neighbor opposition is a common risk factor.
            )
        elif zoning_key == "unknown":
            warnings.append(
                "Zoning classification could not be determined from available data.
                "Manual review of county/municipal zoning records required."
            )

        score = self.clamp_score(score_base)

        summary = (
            f"The site is classified as {zoning_label} based on {data_source} data. "
            f"Permit pathway: {permit_path}. "
        )
        if score >= 75:
            summary += "Zoning is favorable for vertiport development with standard
        elif score >= 40:
            summary += "Conditional Use Permit will be required; public hearing and
        else:
            summary += "Zoning presents a significant barrier; rezoning or variance

        return AgentResult(
            score=score,
            summary=summary,
            warnings=warnings,
            raw_data=raw_data,
            recommendations=[
                f"Verify zoning classification with {self._guess_county()} county/city
                "Confirm whether heliport/vertiport is a permitted use or requires
                "Engage local land use attorney before filing permit applications",
            ],
        )

    def _guess_county(self) -> str:
        """Placeholder — in production, reverse geocode to county."""
        return "local"
```

5.4 Agent 3 — Power Grid Agent

```
# backend/app/agents/agent_power.py
"""
Power Grid Agent — assesses electrical grid capacity and interconnection pathway.

Data sources:
  - EIA API v2 (utility service territories):
```

```
      https://api.eia.gov/v2/electricity/operating-generator-capacity/data/
  - NREL Utility Rate Database (URDB):
      https://developer.nrel.gov/api/utility_rates/v3.json
  - OSM Overpass for substation proximity
  - EIA Form 860 (utility interconnection queue) — bulk download
"""
import httpx

from app.agents.base import AgentResult, BaseAgent
from app.config import settings
from app.utils.geo import haversine_miles
from app.utils.retry import async_retry


class PowerGridAgent(BaseAgent):
    name = "power"

    @async_retry(max_attempts=3, base_delay=2.0)
    async def _get_utility_territory(self) -> dict:
        """Query NREL to identify serving utility company."""
        url = "https://developer.nrel.gov/api/utility_rates/v3.json"
        params = {
            "api_key": settings.NREL_API_KEY,
            "lat": self.lat,
            "lon": self.lon,
            "format": "json",
        }
        async with httpx.AsyncClient(timeout=30) as client:
            r = await client.get(url, params=params)
            r.raise_for_status()
            return r.json()

    @async_retry(max_attempts=3, base_delay=2.0)
    async def _get_nearest_substation(self) -> dict:
        """Query OSM Overpass for nearest electrical substation."""
        delta = 0.1  # ~7-mile bounding box
        bbox = f"{self.lat - delta},{self.lon - delta},{self.lat + delta},{self.lon +
        query = f"""
        [out:json][timeout:25];
        (
          node[power=substation]({bbox});
          way[power=substation]({bbox});
        );
        out center body;
        """
        async with httpx.AsyncClient(timeout=30) as client:
            r = await client.post(
                "https://overpass-api.de/api/interpreter",
                data={"data": query},
            )
            r.raise_for_status()
            data = r.json()
            elements = data.get("elements", [])
            if not elements:
                return {"distance_miles": None, "name": None}
```

```
        # Find nearest
        nearest = None
        nearest_dist = float("inf")
        for el in elements:
            lat = el.get("lat") or el.get("center", {}).get("lat", 0)
            lon = el.get("lon") or el.get("center", {}).get("lon", 0)
            dist = haversine_miles(self.lat, self.lon, lat, lon)
            if dist < nearest_dist:
                nearest_dist = dist
                nearest = el

        tags = (nearest or {}).get("tags", {})
        return {
            "distance_miles": round(nearest_dist, 2),
            "name": tags.get("name", "Unnamed substation"),
            "voltage": tags.get("voltage"),
            "operator": tags.get("operator"),
        }

    async def run(self) -> AgentResult:
        warnings = []

        # Fetch utility data
        try:
            utility_data = await self._get_utility_territory()
            utility_name = (
                utility_data.get("outputs", {}).get("utility_name") or "Unknown Utili
            )
            commercial_rate = utility_data.get("outputs", {}).get("commercial") or (
        except Exception as e:
            self.logger.warning(f"NREL utility query failed: {e}")
            utility_name = "Unknown Utility"
            commercial_rate = 0.12

        # Fetch substation proximity
        try:
            substation = await self._get_nearest_substation()
        except Exception as e:
            self.logger.warning(f"OSM substation query failed: {e}")
            substation = {"distance_miles": None, "name": None}

        sub_dist = substation.get("distance_miles")

        raw_data = {
            "utility_name": utility_name,
            "commercial_rate_per_kwh": commercial_rate,
            "nearest_substation": substation,
            "estimated_load_kw": 2000,  # 1 MW per pad × 2 pads minimum
        }

        # Scoring model
        score = 70  # Baseline — most US properties have grid access

        if sub_dist is not None:
            if sub_dist < 0.5:
                score += 20
```

```
        elif sub_dist < 2:
            score += 10
        elif sub_dist > 10:
            score -= 20
            warnings.append(
                f"Nearest substation is {sub_dist:.1f} miles away. "
                "Grid extension likely required; estimated cost $500K-$2M per mi
            )
        elif sub_dist > 5:
            score -= 10
            warnings.append(
                f"Nearest substation is {sub_dist:.1f} miles away. "
                "Distribution upgrade likely required."
            )

    # Commercial rate heuristic for grid congestion
    # High rates often correlate with constrained grids
    if commercial_rate > 0.20:
        score -= 15
        warnings.append(
            f"High commercial electricity rate (${commercial_rate:.2f}/kWh) dete
            "May indicate grid congestion or limited capacity in this region."
        )

    # EV/eVTOL load requirement warning
    warnings.append(
        "eVTOL charging requires 1-2 MW per landing pad. "
        "Utility interconnection agreement will be required. "
        "Expect 12-36 month interconnect queue in congested metros."
    )

    if "PG&E" in utility_name or "Con Edison" in utility_name:
        score -= 15
        warnings.append(
            f"{utility_name} interconnection queue is currently 24+ months. "
            "Grid upgrade costs may reach $8-16M for MW-scale loads."
        )

    score = self.clamp_score(score)

    summary = (
        f"The site is served by {utility_name} with a commercial electricity rat
        f"${commercial_rate:.3f}/kWh. "
    )
    if sub_dist is not None:
        summary += f"The nearest substation is {sub_dist:.1f} miles away. "
    summary += (
        "eVTOL operations require 1-2 MW of charging capacity per landing pad, "
        "necessitating a formal utility interconnection agreement."
    )

    return AgentResult(
        score=score,
        summary=summary,
        warnings=warnings,
        raw_data=raw_data,
```

```
            recommendations=[
                f"Contact {utility_name} to initiate interconnection pre-application
                "Budget 12-36 months for utility interconnection queue",
                "Evaluate on-site battery storage to reduce peak demand charges",
                "Consider solar + BESS microgrid to offset grid upgrade costs",
            ],
        )
```

5.5 Agent 4 — Structural Agent

```
# backend/app/agents/agent_structural.py
"""
Structural Agent — rules-based assessment of structural suitability.

Standards referenced:
  - FAA Engineering Brief 105A (EB 105A): Vertiport Design Standards
  - FAA Advisory Circular AC 150/5390-2D: Heliport Design
  - IBC 2021: International Building Code structural load requirements
  - ASCE 7-22: Minimum Design Loads for Buildings

No external API — uses property metadata provided at submission time.
"""
from app.agents.base import AgentResult, BaseAgent


class StructuralAgent(BaseAgent):
    name = "structural"

    # FAA EB 105A requires TLOF to support 150% MTOW dynamic loads
    # eVTOL aircraft MTOW range: 2,000-7,000 lbs (1,000-3,175 kg)
    # Required live load: 150% × 7,000 lbs = 10,500 lbs = ~100 psf on TLOF
    REQUIRED_LIVE_LOAD_PSF = 100

    async def run(self) -> AgentResult:
        property_type = self.context.get("property_type", "unknown")
        build_year = self.context.get("build_year") or 0
        num_stories = self.context.get("num_stories") or 1

        warnings = []
        score = 70  # Baseline

        raw_data = {
            "property_type": property_type,
            "build_year": build_year,
            "num_stories": num_stories,
            "faa_eb_105a_tlof_live_load_psf": self.REQUIRED_LIVE_LOAD_PSF,
            "faa_eb_105a_fato_size_ft": 40,  # Minimum FATO: 1.5× largest aircraft D-
        }

        # Property type adjustments
        if property_type == "ground":
            score = 95
            summary_base = "Ground-level pad requires minimal structural modificati
        elif property_type == "airport":
            score = 90
```

```
        summary_base = "Airport property likely has existing aviation-grade hard
    elif property_type == "rooftop":
        score = 55
        summary_base = "Rooftop installation requires structural engineering ass
        warnings.append(
            "Rooftop vertiport requires full structural engineering assessment.
            "Estimated cost: $15,000-$50,000 for assessment, $200K-$2M for reinfo
        )
    else:
        score = 60
        summary_base = "Property type unknown; structural assessment required."

    # Build year adjustments (critical for rooftop)
    if build_year > 0:
        if build_year < 1980 and property_type == "rooftop":
            score -= 30
            warnings.append(
                f"Pre-1980 construction (built {build_year}). "
                "Pre-IBC structural codes did not account for aviation live loads
                "Structural reinforcement is very likely required; budget $500K-
            )
        elif build_year < 1994 and property_type == "rooftop":
            score -= 15
            warnings.append(
                f"Building constructed in {build_year} (pre-IBC 2000). "
                "Structural review for 100 psf live load compliance required."
            )
        elif build_year >= 2010 and property_type == "rooftop":
            score += 10
            # Modern buildings designed for heavier mechanical loads
    elif property_type == "rooftop":
        warnings.append(
            "Build year unknown. Structural engineering assessment mandatory befo
        )

    # Height penalties (evacuation route, wind loading, crane access)
    if num_stories > 30:
        score -= 15
        warnings.append(
            f"{num_stories}-story building: High-rise structural assessment requi
            "Wind load analysis per ASCE 7-22 mandatory. Crane access for constru
        )
    elif num_stories > 10 and property_type == "rooftop":
        score -= 5
        warnings.append(
            f"{num_stories}-story rooftop: Elevator shaft access for passengers a
            "Fire suppression per NFPA 418 required for elevated helipad."
        )

    raw_data["estimated_score"] = score
    raw_data["structural_assessment_required"] = (
        property_type == "rooftop" or build_year < 1980
    )

    score = self.clamp_score(score)
```

```
        summary = summary_base + (
            f" The property ({property_type}, built {build_year or 'unknown'}, "
            f"{num_stories} stories) must meet FAA EB 105A requirements: "
            f"{self.REQUIRED_LIVE_LOAD_PSF} psf TLOF live load and minimum 40 ft FATO
        )

        return AgentResult(
            score=score,
            summary=summary,
            warnings=warnings,
            raw_data=raw_data,
            recommendations=[
                "Engage a licensed structural engineer with aviation helipad experie
                "Review FAA Engineering Brief 105A for TLOF/FATO design requirements
                "Reference NFPA 418 for rooftop heliport fire suppression standards"
                "Budget $15K-$50K for structural assessment prior to permitting",
            ],
        )
```

5.6 Agent 5 — Regulatory Agent

```
# backend/app/agents/agent_regulatory.py
"""
Regulatory Agent — static ruleset database for state-specific requirements.

Data source: SkyBase internal regulatory database (maintained quarterly).
References:
  - FAA Engineering Brief 105A
  - NFPA 418 (Standard for Roof-Top Heliports)
  - State aviation authority regulations for FL, TX, NY, CA, IL
  - FDOT Aviation Office (FL only): https://www.fdot.gov/aviation/
"""
from app.agents.base import AgentResult, BaseAgent

# Regulatory profiles by US state
# Each entry: (base_score, requirements, estimated_timeline_months, fast_track_avai
STATE_REGS = {
    "FL": {
        "score": 80,
        "requirements": [
            "FDOT Aviation Office Site Approval Order (SAO) required",
            "FAA Form 7480-1 (Notice of Proposed Construction) — 45 days before cons
            "Florida Building Code (FBC) compliance",
            "Miami-Dade, Broward, or Palm Beach county-specific zoning approval",
            "FDOT District Office pre-application meeting recommended",
        ],
        "timeline_months": "4-8",
        "fast_track": True,
        "fast_track_note": "FDOT offers streamlined approval for compliant vertiport
        "state_agency": "FDOT Aviation Office",
        "state_agency_url": "https://www.fdot.gov/aviation/",
    },
    "TX": {
        "score": 70,
        "requirements": [
```

```
            "TxDOT Aviation Division approval for public-use heliports",
            "FAA Form 7480-1 mandatory",
            "Municipal zoning approval (varies by city)",
            "Texas Commission on Environmental Quality (TCEQ) noise review if near r
        ],
        "timeline_months": "6-12",
        "fast_track": False,
        "state_agency": "TxDOT Aviation Division",
        "state_agency_url": "https://www.txdot.gov/aviation.html",
    },
    "CA": {
        "score": 45,
        "requirements": [
            "Caltrans Division of Aeronautics approval (Cal PUC §21661)",
            "CEQA (California Environmental Quality Act) review — often triggers EIR'
            "FAA Form 7480-1 mandatory",
            "Local ALUC (Airport Land Use Commission) consistency review",
            "Coastal Commission review if within coastal zone",
        ],
        "timeline_months": "18-36",
        "fast_track": False,
        "state_agency": "Caltrans Division of Aeronautics",
        "state_agency_url": "https://dot.ca.gov/programs/aeronautics",
    },
    "NY": {
        "score": 40,
        "requirements": [
            "NYSDOT Bureau of Aviation approval",
            "NYC: DOT Heliport Permit + NYPD permit for NYC sites",
            "FAA Form 7480-1 mandatory",
            "State Environmental Quality Review Act (SEQRA) review",
            "Community Board review (NYC)",
        ],
        "timeline_months": "18-30",
        "fast_track": False,
        "state_agency": "NYSDOT Bureau of Aviation",
        "state_agency_url": "https://www.dot.ny.gov/divisions/operating/oom/transpor
    },
    "IL": {
        "score": 60,
        "requirements": [
            "IDOT Division of Aeronautics site approval",
            "FAA Form 7480-1 mandatory",
            "Chicago: CTA (Chicago Transit Authority) airspace coordination if eleva
            "Illinois EPA review if near industrial sites",
        ],
        "timeline_months": "9-15",
        "fast_track": False,
        "state_agency": "IDOT Division of Aeronautics",
        "state_agency_url": "https://www.idot.illinois.gov/transportation-system/tra
    },
}

UNIVERSAL_REQUIREMENTS = [
    "FAA Form 7480-1 (Notice of Proposed Construction or Alteration) — file 45 days
    "FAA Obstruction Evaluation / Airport Airspace Analysis (OE/AAA)",
```

```
    "NFPA 418 compliance (rooftop heliports) or NFPA 407 (fueling, if applicable)",
    "Fire suppression system per local fire code",
    "Wind sock / aviation lighting per FAA AC 150/5390-2D",
    "Emergency response plan (coordination with local fire department)",
]


class RegulatoryAgent(BaseAgent):
    name = "regulatory"

    def _detect_state(self) -> str:
        """Detect US state from address string."""
        address_upper = self.address.upper()
        for state in STATE_REGS:
            if f", {state}" in address_upper or f" {state} " in address_upper:
                return state
        return "UNKNOWN"

    async def run(self) -> AgentResult:
        warnings = []
        state = self._detect_state()
        state_data = STATE_REGS.get(state)

        raw_data = {
            "detected_state": state,
            "universal_requirements": UNIVERSAL_REQUIREMENTS,
        }

        if state_data:
            base_score = state_data["score"]
            state_requirements = state_data["requirements"]
            timeline = state_data["timeline_months"]
            raw_data.update({
                "state_requirements": state_requirements,
                "estimated_permitting_months": timeline,
                "state_agency": state_data.get("state_agency"),
                "fast_track_available": state_data.get("fast_track", False),
            })

            if state_data.get("fast_track"):
                warnings.append(
                    f"OPPORTUNITY: {state_data.get('fast_track_note', 'Fast-track appr
                )
            else:
                warnings.append(
                    f"No fast-track approval pathway available in {state}. "
                    f"Estimated permitting timeline: {timeline} months."
                )

            if state == "CA":
                warnings.append(
                    "CEQA review is mandatory in California and frequently triggers a
                    "Environmental Impact Report (EIR). Budget 18-36 months and $200K
                    "for permitting in California."
                )
            elif state == "NY":
```

```
            warnings.append(
                "NYC sites require Community Board approval and NYPD heliport pe
                "addition to state requirements. Expect significant community op
                "noise-sensitive neighborhoods."
            )
        else:
            base_score = 55
            state_requirements = []
            warnings.append(
                f"State-specific regulations for this location not in SkyBase databas
                "Manual review of state aviation authority requirements required."
            )
            raw_data["state_requirements"] = []
            raw_data["estimated_permitting_months"] = "12-24"

        # Universal FAA requirements
        warnings.append(
            "FAA Form 7480-1 must be filed at least 45 days before construction begir
            "Failure to file is a federal violation."
        )

        score = self.clamp_score(base_score)

        all_requirements = UNIVERSAL_REQUIREMENTS + state_requirements
        raw_data["all_requirements"] = all_requirements

        summary = (
            f"This site in {state if state != 'UNKNOWN' else 'an unidentified state'}
            f"requires {len(all_requirements)} distinct regulatory approvals. "
            f"Estimated permitting timeline: {state_data['timeline_months'] if state
        )
        if state == "FL":
            summary += "Florida offers the most streamlined vertiport approval pathw
        elif state == "CA":
            summary += "California's CEQA process is the most significant regulatory

        return AgentResult(
            score=score,
            summary=summary,
            warnings=warnings,
            raw_data=raw_data,
            recommendations=[
                "Engage a local aviation attorney before submitting any permit appli
                "File FAA Form 7480-1 as early as possible — 45-day minimum notice re
                f"Contact {state_data['state_agency'] if state_data else 'state aviati
                "Budget for CEQA/SEQRA environmental review if required by state",
            ],
        )
```

5.7 Agent 6 — Cost Estimation Agent

```
# backend/app/agents/agent_cost.py
"""
Cost Estimation Agent — internal cost model based on industry research.

No external API calls. Uses property context + regulatory and structural
agent results (if available) to produce low/mid/high cost estimates.

Cost benchmarks (2024 research):
  - Rooftop helipad retrofit: $150K-$2M
  - Standalone ground vertipad: $800K-$8M
  - Airport-integrated vertiport: $8M-$15M
  - Full vertihub (multi-pad, terminal): $30M-$150M
  - Charging infrastructure: $2.1M-$4M per MW installed
  - eVTOL charging: 1-2 MW per landing pad
  - Structural assessment: $15K-$50K
  - Permitting + legal: $50K-$500K depending on state
"""
from app.agents.base import AgentResult, BaseAgent


COST_MODELS = {
    "rooftop": {
        "label": "Rooftop Vertipad",
        "low": 150_000,
        "mid": 750_000,
        "high": 2_000_000,
        "structural_required": True,
        "description": "Rooftop conversion with structural reinforcement, FATO, ligl
    },
    "ground": {
        "label": "Standalone Ground Vertiport",
        "low": 800_000,
        "mid": 3_000_000,
        "high": 8_000_000,
        "structural_required": False,
        "description": "New ground pad with terminal, charging stations, access cont
    },
    "airport": {
        "label": "Airport-Integrated Vertiport",
        "low": 8_000_000,
        "mid": 11_000_000,
        "high": 15_000_000,
        "structural_required": False,
        "description": "Full airport integration with terminal link, dedicated airs
    },
    "unknown": {
        "label": "Unknown Type",
        "low": 500_000,
        "mid": 3_500_000,
        "high": 12_000_000,
        "structural_required": True,
        "description": "Cost estimate requires property type clarification",
    },
}
```

```
# Charging infrastructure cost per MW
CHARGING_COST_PER_MW_LOW = 2_100_000
CHARGING_COST_PER_MW_HIGH = 4_000_000
MW_PER_PAD = 1.5  # Average 1-2 MW per landing pad
DEFAULT_PADS = 2

# Landing fee revenue model
LANDING_FEE_LOW = 10    # $/landing (economy)
LANDING_FEE_MID = 20    # $/landing (standard)
LANDING_FEE_HIGH = 30   # $/landing (premium)
LANDINGS_PER_DAY_ESTIMATE = 20   # Conservative for a 2-pad facility


class CostAgent(BaseAgent):
    name = "cost"

    async def run(self) -> AgentResult:
        warnings = []
        property_type = self.context.get("property_type", "unknown")
        build_year = self.context.get("build_year") or 0
        num_pads = DEFAULT_PADS

        model = COST_MODELS.get(property_type, COST_MODELS["unknown"])

        # Base construction costs
        base_low = model["low"]
        base_mid = model["mid"]
        base_high = model["high"]

        # Add charging infrastructure
        charging_low = CHARGING_COST_PER_MW_LOW * MW_PER_PAD * num_pads
        charging_high = CHARGING_COST_PER_MW_HIGH * MW_PER_PAD * num_pads
        charging_mid = (charging_low + charging_high) / 2

        # Structural uplift for old rooftop buildings
        structural_uplift_low = 0
        structural_uplift_high = 0
        if model["structural_required"] and build_year > 0 and build_year < 1994:
            structural_uplift_low = 200_000
            structural_uplift_high = 2_000_000
            warnings.append(
                f"Pre-1994 construction detected. Structural reinforcement budget: "
                f"${structural_uplift_low:,.0f}-${structural_uplift_high:,.0f}."
            )

        # Permitting costs (state-dependent; use mid-range estimate)
        permitting_low = 50_000
        permitting_high = 500_000
        permitting_mid = 150_000

        # Total cost ranges
        total_low = base_low + charging_low + structural_uplift_low + permitting_low
        total_mid = base_mid + charging_mid + permitting_mid
        total_high = base_high + charging_high + structural_uplift_high + permitting_
```

```
        # ROI analysis
        annual_landings = LANDINGS_PER_DAY_ESTIMATE * 365
        annual_revenue_low = annual_landings * LANDING_FEE_LOW
        annual_revenue_mid = annual_landings * LANDING_FEE_MID
        annual_revenue_high = annual_landings * LANDING_FEE_HIGH

        payback_mid_years = total_mid / annual_revenue_mid if annual_revenue_mid > 0

        # Scoring: inversely proportional to total_mid relative to thresholds
        if total_mid < 500_000:
            score = 95
        elif total_mid < 2_000_000:
            score = 80
        elif total_mid < 5_000_000:
            score = 65
        elif total_mid < 10_000_000:
            score = 45
        elif total_mid < 20_000_000:
            score = 30
        else:
            score = 15
            warnings.append(
                f"Estimated total cost exceeds $20M (mid estimate: ${total_mid:,.0f}
                "Project economics require significant traffic volume or ancillary r
            )

        if payback_mid_years and payback_mid_years > 20:
            warnings.append(
                f"Estimated payback period of {payback_mid_years:.0f} years at ${LAND
                "exceeds typical investment horizon. Consider premium pricing or re
            )
            score -= 10

        raw_data = {
            "property_type": property_type,
            "model_label": model["label"],
            "num_pads": num_pads,
            "cost_breakdown": {
                "construction_low": base_low,
                "construction_mid": base_mid,
                "construction_high": base_high,
                "charging_infrastructure_low": charging_low,
                "charging_infrastructure_high": charging_high,
                "structural_reinforcement_low": structural_uplift_low,
                "structural_reinforcement_high": structural_uplift_high,
                "permitting_legal_low": permitting_low,
                "permitting_legal_high": permitting_high,
            },
            "total_project_cost": {
                "low": total_low,
                "mid": total_mid,
                "high": total_high,
            },
            "roi_analysis": {
                "assumed_landing_fee_low": LANDING_FEE_LOW,
                "assumed_landing_fee_mid": LANDING_FEE_MID,
```

```
                "assumed_landing_fee_high": LANDING_FEE_HIGH,
                "assumed_landings_per_day": LANDINGS_PER_DAY_ESTIMATE,
                "annual_revenue_low": annual_revenue_low,
                "annual_revenue_mid": annual_revenue_mid,
                "annual_revenue_high": annual_revenue_high,
                "payback_period_years_mid": round(payback_mid_years, 1) if payback_m
            },
        }

        score = self.clamp_score(score)

        summary = (
            f"Estimated total project cost for a {model['label']}: "
            f"${total_low:,.0f}-${total_high:,.0f} (mid: ${total_mid:,.0f}). "
            f"At ${LANDING_FEE_MID}/landing and {LANDINGS_PER_DAY_ESTIMATE} landings/d
            f"estimated payback is {payback_mid_years:.1f} years." if payback_mid_ye
            f"Estimated total project cost: ${total_low:,.0f}-${total_high:,.0f}."
        )

        return AgentResult(
            score=score,
            summary=summary,
            warnings=warnings,
            raw_data=raw_data,
            recommendations=[
                "Obtain three contractor bids for construction cost verification",
                "Model landing fee sensitivity at $10, $20, $30, and $50/landing",
                "Explore FDOT/FAA grant funding to offset capital costs",
                "Consider phased development: start with 1 pad, expand based on dema
            ],
        )
```

5.8 Agent 7 — Environmental/Noise Agent

```
# backend/app/agents/agent_noise.py
"""
Noise / Environmental Agent — assesses noise conflicts and environmental constraints

Data sources:
  - NOAA Aviation Weather Center (AWC) METAR data for wind patterns:
      https://aviationweather.gov/api/data/metar
  - OpenFEMA API (flood zone lookup):
      https://www.fema.gov/api/open/v1/nfhlByAddress
  - EPA EJScreen API (environmental justice screening):
      https://ejscreen.epa.gov/mapper/ejscreenRESTbroker.aspx
  - OSM Overpass (noise-sensitive uses: schools, hospitals, parks)
"""
import httpx

from app.agents.base import AgentResult, BaseAgent
from app.utils.geo import haversine_miles
from app.utils.retry import async_retry


NOISE_SENSITIVE_TAGS = {
```

```
    "amenity": ["school", "college", "university", "hospital", "clinic", "place_of_
    "leisure": ["park", "playground", "nature_reserve"],
}

FLOOD_ZONE_SCORES = {
    "A": -30,   # 100-year flood zone
    "AE": -30,  # 100-year flood zone with BFE
    "AH": -25,
    "AO": -25,
    "V": -40,   # Coastal high-hazard
    "VE": -40,
    "X": 0,     # Minimal flood hazard
    "D": -10,   # Undetermined
}


class NoiseAgent(BaseAgent):
    name = "noise"

    @async_retry(max_attempts=3, base_delay=2.0)
    async def _get_metar_wind(self) -> dict:
        """Get wind data from nearest METAR station via NOAA AWC."""
        url = "https://aviationweather.gov/api/data/metar"
        params = {
            "ids": "",
            "format": "json",
            "bbox": f"{self.lat - 1},{self.lon - 1},{self.lat + 1},{self.lon + 1}",
            "hours": 24,
        }
        async with httpx.AsyncClient(timeout=20) as client:
            r = await client.get(url, params=params)
            r.raise_for_status()
            data = r.json()
            if not data:
                return {}
            # Average wind direction and speed from last 24h METARs
            directions = [m.get("wdir", 0) for m in data if m.get("wdir")]
            speeds = [m.get("wspd", 0) for m in data if m.get("wspd")]
            return {
                "station_count": len(data),
                "avg_wind_direction_deg": round(sum(directions) / len(directions)) i
                "avg_wind_speed_kt": round(sum(speeds) / len(speeds), 1) if speeds e
                "nearest_station": data[0].get("station") if data else None,
            }

    @async_retry(max_attempts=3, base_delay=2.0)
    async def _get_flood_zone(self) -> dict:
        """Query OpenFEMA for flood zone classification."""
        url = "https://www.fema.gov/api/open/v1/nfhlByAddress"
        params = {
            "Address": self.address,
            "format": "json",
        }
        async with httpx.AsyncClient(timeout=25) as client:
            r = await client.get(url, params=params)
            if r.status_code == 200:
```

```
            data = r.json()
            zones = data.get("floodZones", [])
            if zones:
                return {"zone": zones[0].get("fldZone", "X"), "source": "FEMA NF
    return {"zone": "X", "source": "default_fallback"}

@async_retry(max_attempts=3, base_delay=2.0)
async def _get_noise_sensitive_uses(self) -> list[dict]:
    """Query OSM for nearby noise-sensitive locations within 0.5 miles."""
    delta = 0.008  # ~0.55 mile
    bbox = f"{self.lat - delta},{self.lon - delta},{self.lat + delta},{self.lon +
    tag_filters = "\n".join(
        f'  node[amenity={v}]({bbox});'
        for v in NOISE_SENSITIVE_TAGS["amenity"]
    ) + "\n" + "\n".join(
        f'  node[leisure={v}]({bbox});'
        for v in NOISE_SENSITIVE_TAGS["leisure"]
    )
    query = f"[out:json][timeout:25];\n(\n{tag_filters}\n);\nout body;"

    async with httpx.AsyncClient(timeout=30) as client:
        r = await client.post(
            "https://overpass-api.de/api/interpreter",
            data={"data": query},
        )
        r.raise_for_status()
        elements = r.json().get("elements", [])
        results = []
        for el in elements:
            dist = haversine_miles(
                self.lat, self.lon,
                el.get("lat", 0), el.get("lon", 0),
            )
            tags = el.get("tags", {})
            results.append({
                "name": tags.get("name", "Unnamed"),
                "type": tags.get("amenity") or tags.get("leisure"),
                "distance_miles": round(dist, 3),
            })
        results.sort(key=lambda x: x["distance_miles"])
        return results

async def run(self) -> AgentResult:
    warnings = []

    # Wind data
    try:
        wind = await self._get_metar_wind()
    except Exception as e:
        self.logger.warning(f"METAR query failed: {e}")
        wind = {}

    # Flood zone
    try:
        flood = await self._get_flood_zone()
    except Exception as e:
```

```
            self.logger.warning(f"FEMA query failed: {e}")
            flood = {"zone": "X", "source": "unavailable"}

        # Noise-sensitive uses
        try:
            sensitive_uses = await self._get_noise_sensitive_uses()
        except Exception as e:
            self.logger.warning(f"OSM noise query failed: {e}")
            sensitive_uses = []

        score = 75  # Baseline

        # Flood zone scoring
        flood_zone = flood.get("zone", "X")
        flood_penalty = FLOOD_ZONE_SCORES.get(flood_zone[0] if flood_zone else "X",
        score += flood_penalty
        if flood_penalty < -20:
            warnings.append(
                f"FEMA Flood Zone {flood_zone} — site is in a high-hazard flood area.
                "Drainage engineering required; FEMA LOMA may be needed. "
                "Flood insurance costs may be prohibitive."
            )
        elif flood_penalty < 0:
            warnings.append(
                f"FEMA Flood Zone {flood_zone} — some flood risk present. "
                "Review flood mitigation requirements with civil engineer."
            )

        # Noise-sensitive use scoring
        very_close = [u for u in sensitive_uses if u["distance_miles"] < 0.1]  # < 50
        close = [u for u in sensitive_uses if 0.1 <= u["distance_miles"] < 0.25]

        for use in very_close:
            score -= 20
            warnings.append(
                f"{use['type'].title()} ({use['name']}) is {use['distance_miles'] * 52
                "Noise mitigation plan required. Community opposition likely."
            )

        for use in close:
            score -= 10
            warnings.append(
                f"{use['type'].title()} ({use['name']}) is {use['distance_miles']:.2f
                "Noise impact assessment may be required."
            )

        # Wind assessment
        wind_speed = wind.get("avg_wind_speed_kt")
        if wind_speed and wind_speed > 25:
            score -= 10
            warnings.append(
                f"Average wind speed of {wind_speed:.0f} knots detected. "
                "High wind conditions may restrict eVTOL operations and require wind
            )

        raw_data = {
```

```
            "flood_zone": flood,
            "wind_data": wind,
            "noise_sensitive_uses": sensitive_uses[:20],
            "very_close_count": len(very_close),
            "close_count": len(close),
        }

        score = self.clamp_score(score)

        summary = f"Flood zone: {flood_zone}. "
        if sensitive_uses:
            summary += (
                f"{len(sensitive_uses)} noise-sensitive uses found within 0.25 miles
                f"(nearest: {sensitive_uses[0]['type']} at {sensitive_uses[0]['distanc
            )
        else:
            summary += "No noise-sensitive uses detected within 0.25 miles. "

        if wind_speed:
            summary += f"Prevailing winds at {wind_speed:.0f} knots from {wind.get('

        return AgentResult(
            score=score,
            summary=summary,
            warnings=warnings,
            raw_data=raw_data,
            recommendations=[
                "Commission an FAA-compliant noise study per FAA Order 1050.1F",
                f"Verify FEMA flood zone {flood_zone} with civil engineer",
                "Engage community stakeholders early, especially if schools/hospitals
                "Design approach/departure paths to avoid noise-sensitive areas",
            ],
        )
```

6. Agent Orchestration Layer

6.1 Celery Application

```
# backend/app/celery_app.py
from celery import Celery

from app.config import settings

celery_app = Celery(
    "skybase",
    broker=settings.REDIS_URL,
    backend=settings.REDIS_URL,
    include=[
        "app.tasks.task_airspace",
        "app.tasks.task_zoning",
        "app.tasks.task_power",
        "app.tasks.task_structural",
        "app.tasks.task_regulatory",
```

```
        "app.tasks.task_cost",
        "app.tasks.task_noise",
        "app.tasks.task_aggregate",
    ],
)

celery_app.conf.update(
    task_serializer="json",
    accept_content=["json"],
    result_serializer="json",
    timezone="UTC",
    enable_utc=True,
    task_track_started=True,
    task_acks_late=True,          # Acknowledge only after task completes
    worker_prefetch_multiplier=1,  # One task per worker at a time (CPU-bound agents
    task_soft_time_limit=600,      # 10 minutes soft limit
    task_time_limit=900,           # 15 minutes hard kill
    result_expires=86400,          # Keep results 24 hours
    task_routes={
        "app.tasks.task_aggregate.*": {"queue": "aggregation"},
        "app.tasks.*": {"queue": "agents"},
    },
)
```

6.2 Individual Task Wrappers

```
# backend/app/tasks/task_airspace.py
import logging
from datetime import datetime, timezone

from app.agents.agent_airspace import AirspaceAgent
from app.celery_app import celery_app
from app.tasks._task_helpers import update_agent_result, mark_agent_failed

logger = logging.getLogger(__name__)


@celery_app.task(
    name="tasks.run_airspace_agent",
    bind=True,
    max_retries=2,
    default_retry_delay=30,
)
def run_airspace_agent(self, analysis_id: str, lat: float, lon: float, address: str,
    """Celery task wrapper for the Airspace Agent."""
    import asyncio
    update_agent_result(analysis_id, "airspace", status="running")
    try:
        agent = AirspaceAgent(lat=lat, lon=lon, address=address, context=context)
        result = asyncio.run(agent.run())
        update_agent_result(
            analysis_id, "airspace",
            status="complete",
            score=result.score,
            summary=result.summary,
```

```
            warnings=result.warnings,
            raw_data=result.raw_data,
        )
        return {"agent": "airspace", "score": result.score, "status": "complete"}
    except Exception as exc:
        logger.exception(f"Airspace agent failed for analysis {analysis_id}: {exc}")
        try:
            self.retry(exc=exc)
        except self.MaxRetriesExceededError:
            mark_agent_failed(analysis_id, "airspace", str(exc))
            return {"agent": "airspace", "status": "failed", "error": str(exc)}


# Identical pattern for all other agents — abbreviated for brevity:
# task_zoning.py → run_zoning_agent → ZoningAgent
# task_power.py → run_power_agent → PowerGridAgent
# task_structural.py → run_structural_agent → StructuralAgent
# task_regulatory.py → run_regulatory_agent → RegulatoryAgent
# task_cost.py → run_cost_agent → CostAgent
# task_noise.py → run_noise_agent → NoiseAgent
```

6.3 Task Helpers (Synchronous DB Updates)

```
# backend/app/tasks/_task_helpers.py
"""
Synchronous database helpers for Celery workers.
Workers use a regular (non-async) SQLAlchemy session since Celery tasks
run in a standard thread pool, not an async event loop.
"""
import uuid
from datetime import datetime, timezone
from typing import Any, Optional

from sqlalchemy import create_engine, select
from sqlalchemy.orm import Session, sessionmaker

from app.config import settings

# Synchronous engine for Celery workers
_sync_url = settings.DATABASE_URL.replace(
    "postgresql+asyncpg://", "postgresql://", 1
)
sync_engine = create_engine(_sync_url, pool_pre_ping=True, pool_size=5)
SyncSession = sessionmaker(bind=sync_engine, autocommit=False, autoflush=False)


def update_agent_result(
    analysis_id: str,
    agent_name: str,
    status: str,
    score: Optional[int] = None,
    summary: Optional[str] = None,
    warnings: Optional[list] = None,
    raw_data: Optional[dict] = None,
) -> None:
```

```
    from app.models.agent_result import AgentResult, AgentStatus
    from app.models.analysis import Analysis

    analysis_uuid = uuid.UUID(analysis_id)
    with SyncSession() as session:
        result = session.execute(
            select(AgentResult)
            .where(AgentResult.analysis_id == analysis_uuid)
            .where(AgentResult.agent_name == agent_name)
        ).scalar_one_or_none()

        if result is None:
            result = AgentResult(
                analysis_id=analysis_uuid,
                agent_name=agent_name,
            )
            session.add(result)

        result.status = AgentStatus(status)
        if status == "running":
            result.started_at = datetime.now(timezone.utc)
        elif status in ("complete", "failed"):
            result.completed_at = datetime.now(timezone.utc)

        if score is not None:
            result.score = score
        if summary is not None:
            result.summary = summary
        if warnings is not None:
            result.warnings = warnings
        if raw_data is not None:
            result.raw_data = raw_data

        session.commit()


def mark_agent_failed(analysis_id: str, agent_name: str, error_message: str) -> Non
    update_agent_result(
        analysis_id, agent_name, status="failed", raw_data={"error": error_message}
    )
    # Also store the error message
    from app.models.agent_result import AgentResult, AgentStatus
    analysis_uuid = uuid.UUID(analysis_id)
    with SyncSession() as session:
        result = session.execute(
            select(AgentResult)
            .where(AgentResult.analysis_id == analysis_uuid)
            .where(AgentResult.agent_name == agent_name)
        ).scalar_one_or_none()
        if result:
            result.error_message = error_message
            session.commit()
```

6.4 Orchestrator — Celery Group + Chord

```
# backend/app/tasks/orchestrator.py
"""
Orchestrator dispatches all 7 agents as a Celery group,
then uses a chord callback to trigger PDF aggregation when all complete.

Architecture:
  chord(
    group(agent1, agent2, ..., agent7),
    callback=aggregate_and_generate_report
  ).apply_async()

The chord guarantees the callback fires only after ALL agents in the group
have returned a result (success or failure). Failed agents return a
structured error dict rather than raising, so the chord always completes.
"""
from celery import chain, chord, group

from app.celery_app import celery_app
from app.tasks.task_airspace import run_airspace_agent
from app.tasks.task_zoning import run_zoning_agent
from app.tasks.task_power import run_power_agent
from app.tasks.task_structural import run_structural_agent
from app.tasks.task_regulatory import run_regulatory_agent
from app.tasks.task_cost import run_cost_agent
from app.tasks.task_noise import run_noise_agent
from app.tasks.task_aggregate import aggregate_and_generate_report


def dispatch_analysis(
    analysis_id: str,
    lat: float,
    lon: float,
    address: str,
    property_context: dict,
):
    """
    Dispatch the full analysis pipeline. Returns the chord AsyncResult.
    The chord ID is stored on the Analysis record for progress tracking.
    """
    agent_kwargs = dict(
        analysis_id=analysis_id,
        lat=lat,
        lon=lon,
        address=address,
        context=property_context,
    )

    # All 7 agents run in parallel as a Celery group
    agent_group = group(
        run_airspace_agent.s(**agent_kwargs),
        run_zoning_agent.s(**agent_kwargs),
        run_power_agent.s(**agent_kwargs),
        run_structural_agent.s(**agent_kwargs),
        run_regulatory_agent.s(**agent_kwargs),
```

```
        run_cost_agent.s(**agent_kwargs),
        run_noise_agent.s(**agent_kwargs),
    )

    # Chord: group → callback with all 7 results
    # The callback receives a list of 7 dicts (one per agent task return value)
    pipeline = chord(
        agent_group,
        aggregate_and_generate_report.s(analysis_id=analysis_id),
    )

    # Apply with explicit queue routing
    result = pipeline.apply_async(
        queue="agents",
        soft_time_limit=1800,   # 30 minutes for the full pipeline
    )

    return result
```

6.5 Aggregation Task (Chord Callback)

```
# backend/app/tasks/task_aggregate.py
"""
Aggregation task — chord callback that fires after all 7 agents complete.
Receives a list of agent result dicts. Generates the PDF and sends the email.
"""
import logging
import uuid
from datetime import datetime, timezone

from sqlalchemy import select
from sqlalchemy.orm import selectinload

from app.celery_app import celery_app
from app.models.analysis import Analysis, AnalysisStatus
from app.models.agent_result import AgentResult
from app.services.pdf_service import generate_report_pdf
from app.services.storage_service import upload_pdf_to_storage
from app.services.email_service import send_report_email
from app.tasks._task_helpers import SyncSession

logger = logging.getLogger(__name__)


@celery_app.task(
    name="tasks.aggregate_and_generate_report",
    bind=True,
    queue="aggregation",
    max_retries=1,
)
def aggregate_and_generate_report(
    self,
    agent_results: list[dict],   # List of return values from the group
    analysis_id: str,
) -> dict:
```

```
    """
    Called by Celery chord after all agent tasks complete (success or failure).

    agent_results: list of dicts like:
      [{"agent": "airspace", "score": 72, "status": "complete"}, ...]
      or {"agent": "zoning", "status": "failed", "error": "..."} on failure
    """
    logger.info(f"Aggregation started for analysis {analysis_id}")
    analysis_uuid = uuid.UUID(analysis_id)

    with SyncSession() as session:
        # Load analysis with all agent results
        analysis = session.execute(
            select(Analysis)
            .options(selectinload(Analysis.agent_results))
            .where(Analysis.id == analysis_uuid)
        ).scalar_one()

        if analysis is None:
            logger.error(f"Analysis {analysis_id} not found in aggregation task")
            return {"status": "error", "detail": "analysis_not_found"}

        # Log any agent failures
        failed_agents = [r for r in agent_results if r.get("status") == "failed"]
        if failed_agents:
            logger.warning(
                f"Analysis {analysis_id}: {len(failed_agents)} agents failed: "
                f"{[r['agent'] for r in failed_agents]}"
            )

        try:
            # Generate PDF from all agent results
            pdf_bytes = generate_report_pdf(
                analysis=analysis,
                agent_results=analysis.agent_results,
            )

            # Upload to S3 / Cloudflare R2
            object_key = f"reports/{analysis_id}/vertiport-readiness-report.pdf"
            upload_pdf_to_storage(pdf_bytes, object_key)

            # Update analysis record
            analysis.status = AnalysisStatus.COMPLETE
            analysis.report_url = object_key
            analysis.completed_at = datetime.now(timezone.utc)
            session.commit()

            # Send delivery email
            send_report_email(
                to_email=analysis.user_email,
                analysis_id=str(analysis_id),
                address=analysis.address_formatted or analysis.address_input,
                overall_score=analysis.overall_score,
            )

            logger.info(f"Analysis {analysis_id} complete. Report uploaded to {objec
```

```
            return {"status": "complete", "report_url": object_key}

        except Exception as exc:
            logger.exception(f"Aggregation failed for analysis {analysis_id}: {exc}"
            analysis.status = AnalysisStatus.FAILED
            session.commit()
            raise
```

7. PDF Report Structure (WeasyPrint)

7.1 PDF Service

```
# backend/app/services/pdf_service.py
"""
PDF generation service using WeasyPrint (primary) with ReportLab fallback.
"""
import logging
import os
from pathlib import Path
from typing import Optional

logger = logging.getLogger(__name__)

TEMPLATES_DIR = Path(__file__).parent.parent / "templates"


def generate_report_pdf(analysis, agent_results: list) -> bytes:
    """
    Generate the full Vertiport Readiness Report PDF.
    Returns raw PDF bytes.
    """
    try:
        return _generate_weasyprint(analysis, agent_results)
    except ImportError:
        logger.warning("WeasyPrint unavailable, falling back to ReportLab")
        return _generate_reportlab(analysis, agent_results)


def _generate_weasyprint(analysis, agent_results: list) -> bytes:
    from weasyprint import HTML, CSS

    html_content = _render_report_html(analysis, agent_results)

    # Load base CSS
    css_path = TEMPLATES_DIR / "report.css"
    css = CSS(filename=str(css_path))

    pdf_bytes = HTML(string=html_content, base_url=str(TEMPLATES_DIR)).write_pdf(
        stylesheets=[css],
        optimize_images=True,
        presentational_hints=True,
    )
    return pdf_bytes
```

```python
def _render_report_html(analysis, agent_results: list) -> str:
    """Render the Jinja2 HTML template for the report."""
    from jinja2 import Environment, FileSystemLoader

    env = Environment(
        loader=FileSystemLoader(str(TEMPLATES_DIR)),
        autoescape=True,
    )
    env.filters["currency"] = lambda v: f"${v:,.0f}" if v else "N/A"
    env.filters["score_class"] = _score_to_class

    template = env.get_template("report.html.j2")

    agent_map = {r.agent_name: r for r in agent_results}
    overall_score = analysis.overall_score

    return template.render(
        analysis=analysis,
        agent_map=agent_map,
        overall_score=overall_score,
        score_label=_score_label(overall_score),
        score_color=_score_color(overall_score),
        all_warnings=_collect_warnings(agent_results),
        regulatory_checklist=_build_regulatory_checklist(agent_map),
        top_recommendations=_top_recommendations(agent_results),
        generated_date=analysis.completed_at,
    )


def _score_to_class(score: Optional[int]) -> str:
    if score is None:
        return "score-unknown"
    if score >= 75:
        return "score-high"
    if score >= 50:
        return "score-medium"
    return "score-low"


def _score_label(score: Optional[int]) -> str:
    if score is None:
        return "Incomplete"
    if score >= 75:
        return "High Readiness"
    if score >= 50:
        return "Moderate Readiness"
    if score >= 25:
        return "Low Readiness"
    return "Not Recommended"


def _score_color(score: Optional[int]) -> str:
    if score is None:
        return "#9CA3AF"
```

```
    if score >= 75:
        return "#10B981"
    if score >= 50:
        return "#F59E0B"
    return "#EF4444"


def _collect_warnings(agent_results: list) -> list[dict]:
    warnings = []
    for r in agent_results:
        for w in (r.warnings or []):
            warnings.append({"agent": r.agent_name, "warning": w})
    return warnings


def _build_regulatory_checklist(agent_map: dict) -> list[dict]:
    """Build the regulatory checklist from the regulatory agent's raw data."""
    reg = agent_map.get("regulatory")
    if not reg or not reg.raw_data:
        return []
    return [
        {"item": req, "checked": False}
        for req in reg.raw_data.get("all_requirements", [])
    ]


def _top_recommendations(agent_results: list, n: int = 5) -> list[str]:
    """Collect the highest-priority recommendations across all agents."""
    all_recs = []
    for r in sorted(agent_results, key=lambda x: x.score or 50):
        # Lower-scoring agents' recommendations are highest priority
        recs = (r.raw_data or {}).get("recommendations", [])
        all_recs.extend(recs)
    return all_recs[:n]


def _generate_reportlab(analysis, agent_results: list) -> bytes:
    """Fallback PDF generation using ReportLab."""
    from reportlab.lib.pagesizes import letter
    from reportlab.lib.styles import getSampleStyleSheet
    from reportlab.platypus import (
        Paragraph, SimpleDocTemplate, Spacer, Table, TableStyle,
    )
    from reportlab.lib import colors
    import io

    buffer = io.BytesIO()
    doc = SimpleDocTemplate(buffer, pagesize=letter)
    styles = getSampleStyleSheet()
    story = []

    story.append(Paragraph(
        "SkyBase Vertiport Readiness Report",
        styles["Title"],
    ))
    story.append(Paragraph(
```

```
        f"Property: {analysis.address_formatted or analysis.address_input}",
        styles["Heading2"],
    ))
    story.append(Paragraph(
        f"Overall Score: {analysis.overall_score or 'N/A'} / 100",
        styles["Heading1"],
    ))
    story.append(Spacer(1, 12))

    # Agent score table
    table_data = [["Agent", "Score", "Status"]]
    for r in agent_results:
        table_data.append([
            r.agent_name.title(),
            str(r.score or "N/A"),
            r.status,
        ])

    t = Table(table_data)
    t.setStyle(TableStyle([
        ("BACKGROUND", (0, 0), (-1, 0), colors.grey),
        ("TEXTCOLOR", (0, 0), (-1, 0), colors.whitesmoke),
        ("GRID", (0, 0), (-1, -1), 0.5, colors.black),
        ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, colors.lightgrey]),
    ]))
    story.append(t)

    doc.build(story)
    return buffer.getvalue()
```

7.2 Report HTML Template Structure

```
backend/app/templates/
├── report.html.j2          # Main Jinja2 template
├── report.css              # Print-optimized CSS (page breaks, fonts)
└── partials/
    ├── _cover.html.j2      # Cover page
    ├── _executive_summary.html.j2
    ├── _agent_section.html.j2    # Reusable per-agent section
    ├── _cost_analysis.html.j2
    ├── _regulatory_checklist.html.j2
    └── _next_steps.html.j2
```

report.html.j2 skeleton:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Vertiport Readiness Report — {{ analysis.address_formatted }}</title>
</head>
<body>
```

```
  {# — COVER PAGE — #}
  <div class="page cover-page">
    <div class="logo-block">
      <img src="logo.png" alt="SkyBase Intelligence" class="logo">
    </div>
    <h1 class="report-title">Vertiport Readiness Report</h1>
    <p class="address">{{ analysis.address_formatted }}</p>
    <div class="score-ring" style="--score-color: {{ score_color }};">
      <span class="score-value">{{ overall_score or '-' }}</span>
      <span class="score-max">/100</span>
    </div>
    <p class="score-label" style="color: {{ score_color }};">{{ score_label }}</p>
    <p class="report-date">Generated: {{ generated_date.strftime('%B %d, %Y') if ge
    <p class="report-id">Report ID: {{ analysis.id }}</p>
  </div>
  <div class="page-break"></div>

  {# — EXECUTIVE SUMMARY — #}
  {% include 'partials/_executive_summary.html.j2' %}
  <div class="page-break"></div>

  {# — AGENT SECTIONS (one per agent) — #}
  {% for agent_name in ['airspace', 'zoning', 'power', 'structural', 'regulatory', '
    {% set agent = agent_map.get(agent_name) %}
    {% if agent %}
      {% include 'partials/_agent_section.html.j2' %}
      <div class="page-break"></div>
    {% endif %}
  {% endfor %}

  {# — COST ANALYSIS — #}
  {% include 'partials/_cost_analysis.html.j2' %}
  <div class="page-break"></div>

  {# — REGULATORY CHECKLIST — #}
  {% include 'partials/_regulatory_checklist.html.j2' %}
  <div class="page-break"></div>

  {# — NEXT STEPS — #}
  {% include 'partials/_next_steps.html.j2' %}

</body>
</html>
```

report.css (key print rules):

```
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&dis

* { box-sizing: border-box; margin: 0; padding: 0; }

body {
  font-family: 'Inter', sans-serif;
  font-size: 10pt;
  line-height: 1.5;
  color: #111827;
```

```
}

@page {
  size: Letter;
  margin: 0.75in 0.75in 1in 0.75in;
  @bottom-center {
    content: "SkyBase Intelligence Platform  |  skybaseintel.com  |  Page " counter(
    font-size: 8pt;
    color: #6B7280;
  }
}

.page-break { page-break-after: always; break-after: page; }

/* Cover page */
.cover-page { text-align: center; padding-top: 2in; }
.report-title { font-size: 28pt; font-weight: 700; color: #1E3A5F; margin-bottom: 0.
.address { font-size: 14pt; color: #374151; margin-bottom: 1em; }

/* Score ring */
.score-ring {
  display: inline-block;
  width: 120pt;
  height: 120pt;
  border-radius: 50%;
  border: 8pt solid var(--score-color, #9CA3AF);
  line-height: 104pt;
  margin: 1em auto;
}
.score-value { font-size: 36pt; font-weight: 700; }

/* Agent section */
.agent-section h2 { font-size: 16pt; color: #1E3A5F; border-bottom: 2pt solid #E5E7E
.agent-score-badge {
  display: inline-block;
  padding: 4pt 12pt;
  border-radius: 16pt;
  font-weight: 600;
  font-size: 12pt;
  color: white;
}
.score-high .agent-score-badge { background: #10B981; }
.score-medium .agent-score-badge { background: #F59E0B; }
.score-low .agent-score-badge { background: #EF4444; }

/* Warning boxes */
.warning-box {
  background: #FEF3C7;
  border-left: 4pt solid #F59E0B;
  padding: 8pt 12pt;
  margin: 8pt 0;
  border-radius: 0 4pt 4pt 0;
}
.warning-box.critical {
  background: #FEE2E2;
  border-left-color: #EF4444;
```

```
}

/* Tables */
table { width: 100%; border-collapse: collapse; margin: 8pt 0; }
th { background: #1E3A5F; color: white; padding: 6pt 8pt; text-align: left; font-siz
td { padding: 5pt 8pt; border-bottom: 0.5pt solid #E5E7EB; font-size: 9pt; }
tr:nth-child(even) td { background: #F9FAFB; }

/* Regulatory checklist */
.checklist-item { padding: 4pt 0; display: flex; align-items: flex-start; gap: 8pt;
.checklist-item input[type="checkbox"] { margin-top: 2pt; flex-shrink: 0; }
```

8. Frontend (Next.js)

8.1 TypeScript Types

```
// frontend/src/lib/types.ts

export type AnalysisStatus = "pending" | "processing" | "complete" | "failed";
export type AgentStatus = "pending" | "running" | "complete" | "failed";
export type PaymentStatus = "unpaid" | "paid" | "refunded";

export interface AgentResult {
  id: string;
  agent_name: "airspace" | "zoning" | "power" | "structural" | "regulatory" | "cost'
  status: AgentStatus;
  score: number | null;
  summary: string | null;
  warnings: string[] | null;
  started_at: string | null;
  completed_at: string | null;
  error_message: string | null;
}

export interface Analysis {
  id: string;
  address_input: string;
  address_formatted: string | null;
  latitude: number | null;
  longitude: number | null;
  status: AnalysisStatus;
  payment_status: PaymentStatus;
  report_url: string | null;
  overall_score: number | null;
  created_at: string;
  completed_at: string | null;
  agent_results: AgentResult[];
}

export interface CreateAnalysisRequest {
  address: string;
  email: string;
  property_type: "rooftop" | "ground" | "airport" | "unknown";
```

```
  build_year?: number;
  num_stories?: number;
}

export interface CreateAnalysisResponse {
  analysis_id: string;
  checkout_url: string;
  message: string;
}
```

8.2 API Client

```
// frontend/src/lib/api.ts
const API_BASE = process.env.NEXT_PUBLIC_API_URL ?? "http://localhost:8000";

async function apiFetch<T>(path: string, options?: RequestInit): Promise<T> {
  const res = await fetch(`${API_BASE}${path}`, {
    headers: { "Content-Type": "application/json" },
    ...options,
  });
  if (!res.ok) {
    const error = await res.json().catch(() => ({ detail: res.statusText }));
    throw new Error(error.detail ?? `HTTP ${res.status}`);
  }
  return res.json();
}

export const api = {
  createAnalysis: (body: CreateAnalysisRequest) =>
    apiFetch<CreateAnalysisResponse>("/api/v1/analyses/analyze", {
      method: "POST",
      body: JSON.stringify(body),
    }),

  getAnalysis: (id: string) =>
    apiFetch<Analysis>(`/api/v1/analyses/${id}`),

  getReportUrl: (id: string) =>
    apiFetch<{ download_url: string; expires_in_seconds: number }>(
      `/api/v1/analyses/${id}/report`
    ),
};
```

8.3 Landing Page (/)

```
// frontend/src/app/page.tsx
import Link from "next/link";
import { PricingCard } from "@/components/PricingCard";

export default function LandingPage() {
  return (
    <main>
      {/* Hero */}
```

```
      <section className="bg-gradient-to-br from-sky-950 to-blue-800 text-white py-2
        <h1 className="text-5xl font-bold tracking-tight mb-4">
          Know If Your Property Is Vertiport-Ready
        </h1>
        <p className="text-xl text-sky-200 max-w-2xl mx-auto mb-8">
          SkyBase runs 7 parallel AI-powered analysis agents — airspace, zoning,
          power, structural, regulatory, cost, and environmental — and delivers
          a complete Vertiport Readiness Report in 30 minutes.
        </p>
        <Link
          href="/analyze"
          className="bg-white text-sky-900 font-semibold px-8 py-4 rounded-xl
                     text-lg hover:bg-sky-50 transition-colors inline-block"
        >
          Analyze My Property — $499
        </Link>
        <p className="text-sky-300 text-sm mt-3">
          30-minute turnaround • PDF delivered by email • 100% satisfaction guarantee
        </p>
      </section>

      {/* How It Works */}
      <section className="py-20 px-6 max-w-5xl mx-auto">
        <h2 className="text-3xl font-bold text-center text-gray-900 mb-12">
          How It Works
        </h2>
        <div className="grid grid-cols-1 md:grid-cols-3 gap-8">
          {[
            {
              step: "1",
              title: "Enter Your Address",
              desc: "Provide the US property address. We geocode it instantly.",
            },
            {
              step: "2",
              title: "7 Agents Run in Parallel",
              desc: "FAA airspace, zoning, power grid, structural, regulatory, cost,
            },
            {
              step: "3",
              title: "Get Your Report",
              desc: "A 20+ page PDF Vertiport Readiness Report lands in your inbox i
            },
          ].map((item) => (
            <div key={item.step} className="text-center p-6 rounded-2xl bg-gray-50">
              <div className="w-12 h-12 bg-sky-800 text-white rounded-full flex items
                {item.step}
              </div>
              <h3 className="text-lg font-semibold mb-2">{item.title}</h3>
              <p className="text-gray-600 text-sm">{item.desc}</p>
            </div>
          ))}
        </div>
      </section>

      {/* Pricing */}
```

```
      <section className="bg-gray-50 py-20 px-6">
        <h2 className="text-3xl font-bold text-center text-gray-900 mb-12">
          Pricing
        </h2>
        <div className="grid grid-cols-1 md:grid-cols-3 gap-6 max-w-5xl mx-auto">
          <PricingCard
            tier="Single Report"
            price="$499"
            unit="per report"
            features={[
              "Full 7-agent analysis",
              "20+ page PDF report",
              "Email delivery",
              "30-minute turnaround",
            ]}
            cta="Get Started"
            href="/analyze"
          />
          <PricingCard
            tier="Bulk"
            price="$200"
            unit="per report (5+ pack)"
            features={[
              "Everything in Single",
              "API access",
              "Priority queue",
              "White-label option",
            ]}
            cta="Contact Sales"
            href="mailto:sales@skybaseintel.com"
            highlighted
          />
          <PricingCard
            tier="Enterprise API"
            price="$500"
            unit="per month + $150/report"
            features={[
              "REST API access",
              "Webhook delivery",
              "Custom branding",
              "SLA guarantee",
            ]}
            cta="Contact Sales"
            href="mailto:sales@skybaseintel.com"
          />
        </div>
      </section>
    </main>
  );
}
```

8.4 Analyze Page (`/analyze`)

```
// frontend/src/app/analyze/page.tsx
"use client";

import { useState } from "react";
import { useRouter } from "next/navigation";
import { AddressInput } from "@/components/AddressInput";
import { api } from "@/lib/api";
import type { CreateAnalysisRequest } from "@/lib/types";

export default function AnalyzePage() {
  const router = useRouter();
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const [form, setForm] = useState<CreateAnalysisRequest>({
    address: "",
    email: "",
    property_type: "unknown",
  });

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    setLoading(true);
    setError(null);

    try {
      const { checkout_url } = await api.createAnalysis(form);
      // Redirect to Stripe Checkout
      window.location.href = checkout_url;
    } catch (err: any) {
      setError(err.message ?? "An error occurred. Please try again.");
      setLoading(false);
    }
  }

  return (
    <main className="max-w-xl mx-auto py-16 px-6">
      <h1 className="text-3xl font-bold text-gray-900 mb-2">Analyze Your Property</h1
      <p className="text-gray-500 mb-8">
        Enter your property details. You'll be redirected to secure checkout, then
        your 30-minute analysis begins immediately after payment.
      </p>

      <form onSubmit={handleSubmit} className="space-y-5">
        <div>
          <label className="block text-sm font-medium text-gray-700 mb-1">
            Property Address *
          </label>
          <AddressInput
            value={form.address}
            onChange={(addr) => setForm((f) => ({ ...f, address: addr }))}
            placeholder="1234 NW 56th St, Miami, FL 33142"
          />
        </div>
```

```
        <div>
          <label className="block text-sm font-medium text-gray-700 mb-1">
            Your Email *
          </label>
          <input
            type="email"
            required
            value={form.email}
            onChange={(e) => setForm((f) => ({ ...f, email: e.target.value }))}
            className="w-full border border-gray-300 rounded-lg px-3 py-2 focus:outl
            placeholder="you@example.com"
          />
        </div>

        <div>
          <label className="block text-sm font-medium text-gray-700 mb-1">
            Property Type
          </label>
          <select
            value={form.property_type}
            onChange={(e) =>
              setForm((f) => ({ ...f, property_type: e.target.value as any }))
            }
            className="w-full border border-gray-300 rounded-lg px-3 py-2 focus:outl
          >
            <option value="unknown">I'm not sure</option>
            <option value="rooftop">Rooftop</option>
            <option value="ground">Ground level</option>
            <option value="airport">Airport property</option>
          </select>
        </div>

        {form.property_type !== "unknown" && (
          <div className="grid grid-cols-2 gap-4">
            <div>
              <label className="block text-sm font-medium text-gray-700 mb-1">
                Build Year
              </label>
              <input
                type="number"
                min={1800}
                max={2025}
                value={form.build_year ?? ""}
                onChange={(e) =>
                  setForm((f) => ({ ...f, build_year: Number(e.target.value) || unde
                }
                className="w-full border border-gray-300 rounded-lg px-3 py-2"
                placeholder="e.g. 1998"
              />
            </div>
            <div>
              <label className="block text-sm font-medium text-gray-700 mb-1">
                Number of Stories
              </label>
              <input
                type="number"
```

```
                min={1}
                max={200}
                value={form.num_stories ?? ""}
                onChange={(e) =>
                  setForm((f) => ({ ...f, num_stories: Number(e.target.value) || und
                }
                className="w-full border border-gray-300 rounded-lg px-3 py-2"
                placeholder="e.g. 12"
              />
            </div>
          </div>
        )}

        {error && (
          <div className="bg-red-50 border border-red-200 text-red-700 rounded-lg p
            {error}
          </div>
        )}

        <button
          type="submit"
          disabled={loading || !form.address || !form.email}
          className="w-full bg-sky-800 text-white font-semibold py-3 px-6 rounded-xl
                     hover:bg-sky-700 disabled:opacity-50 disabled:cursor-not-allowed
        >
          {loading ? "Creating your analysis..." : "Continue to Payment — $499"}
        </button>

        <p className="text-xs text-gray-400 text-center">
          Secure payment via Stripe. Analysis begins immediately after payment confi
        </p>
      </form>
    </main>
  );
}
```

8.5 Status Page (`/status/[id]`)

```
// frontend/src/app/status/[id]/page.tsx
"use client";

import { useEffect, useState } from "react";
import { useParams, useRouter } from "next/navigation";
import { AnalysisProgress } from "@/components/AnalysisProgress";
import { ScoreGauge } from "@/components/ScoreGauge";
import { api } from "@/lib/api";
import type { Analysis } from "@/lib/types";

const POLL_INTERVAL_MS = 5000;

export default function StatusPage() {
  const { id } = useParams<{ id: string }>();
  const router = useRouter();
  const [analysis, setAnalysis] = useState<Analysis | null>(null);
  const [error, setError] = useState<string | null>(null);
```

```
useEffect(() => {
  if (!id) return;

  const poll = async () => {
    try {
      const data = await api.getAnalysis(id);
      setAnalysis(data);

      if (data.status === "complete") {
        // Navigate to report viewer
        router.push(`/report/${id}`);
      }
    } catch (err: any) {
      setError(err.message);
    }
  };

  poll(); // Initial fetch
  const interval = setInterval(poll, POLL_INTERVAL_MS);
  return () => clearInterval(interval);
}, [id]);

if (error) {
  return (
    <div className="max-w-xl mx-auto py-16 px-6 text-center">
      <p className="text-red-600">{error}</p>
    </div>
  );
}

if (!analysis) {
  return (
    <div className="max-w-xl mx-auto py-16 px-6 text-center">
      <div className="animate-spin h-8 w-8 border-4 border-sky-800 border-t-transp
      <p className="mt-4 text-gray-500">Loading analysis...</p>
    </div>
  );
}

return (
  <main className="max-w-2xl mx-auto py-12 px-6">
    <h1 className="text-2xl font-bold text-gray-900 mb-1">Analyzing Your Property<,
    <p className="text-gray-500 mb-8 text-sm">
      {analysis.address_formatted ?? analysis.address_input}
    </p>

    {analysis.overall_score !== null && (
      <div className="flex justify-center mb-8">
        <ScoreGauge score={analysis.overall_score} />
      </div>
    )}

    <AnalysisProgress agentResults={analysis.agent_results} />

    {analysis.status === "failed" && (
```

```
        <div className="mt-6 bg-red-50 border border-red-200 text-red-700 rounded-x
          Analysis encountered an error. Please contact support@skybaseintel.com
          with your Analysis ID: <code className="font-mono">{id}</code>
        </div>
      )}

      <p className="text-center text-xs text-gray-400 mt-8">
        This page refreshes automatically every 5 seconds.
        Report will appear automatically when complete.
      </p>
    </main>
  );
}
```

8.6 Key Components

```
// frontend/src/components/AnalysisProgress.tsx
"use client";

import type { AgentResult } from "@/lib/types";

const AGENT_LABELS: Record<string, string> = {
  airspace: "Airspace Classification",
  zoning: "Zoning & Land Use",
  power: "Power Grid Capacity",
  structural: "Structural Assessment",
  regulatory: "Regulatory Requirements",
  cost: "Cost Estimation",
  noise: "Environmental & Noise",
};

const AGENT_ORDER = [
  "airspace", "zoning", "power", "structural", "regulatory", "cost", "noise",
];

const STATUS_COLORS: Record<string, string> = {
  pending: "bg-gray-200",
  running: "bg-sky-500 animate-pulse",
  complete: "bg-green-500",
  failed: "bg-red-500",
};

export function AnalysisProgress({ agentResults }: { agentResults: AgentResult[] }) {
  const resultMap = Object.fromEntries(agentResults.map((r) => [r.agent_name, r]));

  return (
    <div className="space-y-4">
      {AGENT_ORDER.map((agentName) => {
        const result = resultMap[agentName];
        const status = result?.status ?? "pending";
        const score = result?.score;

        return (
          <div key={agentName} className="flex items-center gap-4">
            <div className="w-40 text-sm font-medium text-gray-700 flex-shrink-0">
```

```
              {AGENT_LABELS[agentName]}
            </div>
            <div className="flex-1 bg-gray-100 rounded-full h-2.5 overflow-hidden">
              <div
                className={`h-full rounded-full transition-all duration-700 ${STATUS_C
                style={{ width: status === "complete" ? "100%" : status === "running"
              />
            </div>
            <div className="w-20 text-right flex-shrink-0">
              {status === "complete" && score !== null && (
                <span className={`text-sm font-semibold ${
                  score >= 75 ? "text-green-600" : score >= 50 ? "text-amber-600" : "
                }`}>
                  {score}/100
                </span>
              )}
              {status === "running" && (
                <span className="text-xs text-sky-600">Running...</span>
              )}
              {status === "pending" && (
                <span className="text-xs text-gray-400">Queued</span>
              )}
              {status === "failed" && (
                <span className="text-xs text-red-500">Failed</span>
              )}
            </div>
          </div>
        );
      })}
    </div>
  );
}
```

```
// frontend/src/components/ScoreGauge.tsx
"use client";

function scoreColor(score: number): string {
  if (score >= 75) return "#10B981";
  if (score >= 50) return "#F59E0B";
  return "#EF4444";
}

function scoreLabel(score: number): string {
  if (score >= 75) return "High Readiness";
  if (score >= 50) return "Moderate Readiness";
  if (score >= 25) return "Low Readiness";
  return "Not Recommended";
}

export function ScoreGauge({ score }: { score: number }) {
  const color = scoreColor(score);
  const label = scoreLabel(score);
  const circumference = 2 * Math.PI * 54;
  const dash = (score / 100) * circumference;
```

```
  return (
    <div className="flex flex-col items-center">
      <svg width="140" height="140" viewBox="0 0 140 140">
        {/* Background ring */}
        <circle cx="70" cy="70" r="54" fill="none" stroke="#E5E7EB" strokeWidth="10"
        {/* Score arc */}
        <circle
          cx="70"
          cy="70"
          r="54"
          fill="none"
          stroke={color}
          strokeWidth="10"
          strokeDasharray={`${dash} ${circumference}`}
          strokeLinecap="round"
          transform="rotate(-90 70 70)"
          style={{ transition: "stroke-dasharray 1s ease" }}
        />
        <text x="70" y="65" textAnchor="middle" fontSize="28" fontWeight="700" fill="
          {score}
        </text>
        <text x="70" y="82" textAnchor="middle" fontSize="11" fill="#6B7280">
          / 100
        </text>
      </svg>
      <p className="text-sm font-semibold mt-1" style={{ color }}>
        {label}
      </p>
    </div>
  );
}
```

```
// frontend/src/components/AddressInput.tsx
"use client";

import { useEffect, useRef, useState } from "react";

interface Props {
  value: string;
  onChange: (address: string) => void;
  placeholder?: string;
}

export function AddressInput({ value, onChange, placeholder }: Props) {
  const inputRef = useRef<HTMLInputElement>(null);
  const [autocomplete, setAutocomplete] = useState<google.maps.places.Autocomplete

  useEffect(() => {
    if (!window.google || !inputRef.current) return;

    const ac = new window.google.maps.places.Autocomplete(inputRef.current, {
      types: ["address"],
      componentRestrictions: { country: "us" },
      fields: ["formatted_address"],
    });
```

```
    ac.addListener("place_changed", () => {
      const place = ac.getPlace();
      if (place.formatted_address) {
        onChange(place.formatted_address);
      }
    });

    setAutocomplete(ac);
    return () => google.maps.event.clearInstanceListeners(ac);
  }, []);

  return (
    <input
      ref={inputRef}
      type="text"
      value={value}
      onChange={(e) => onChange(e.target.value)}
      placeholder={placeholder}
      className="w-full border border-gray-300 rounded-lg px-3 py-2
                focus:outline-none focus:ring-2 focus:ring-sky-500"
      autoComplete="off"
    />
  );
}
```

9. Payment Flow

9.1 Stripe Service

```
# backend/app/services/stripe_service.py
import stripe

from app.config import settings

stripe.api_key = settings.STRIPE_SECRET_KEY


async def create_checkout_session(
    analysis_id: str,
    email: str,
    metadata: dict,
) -> stripe.checkout.Session:
    """
    Create a Stripe Checkout Session for a $499 one-time payment.

    - mode=payment (not subscription)
    - success_url includes the analysis_id so the frontend can poll status
    - cancel_url returns the user to the analyze page
    - customer_email pre-fills the Stripe checkout form
    - metadata carries analysis_id through to the webhook
    """
    session = stripe.checkout.Session.create(
```

```
        mode="payment",
        payment_method_types=["card"],
        line_items=[
            {
                "price": settings.STRIPE_PRICE_ID,
                "quantity": 1,
            }
        ],
        customer_email=email,
        success_url=(
            f"{settings.FRONTEND_URL}/status/{analysis_id}"
            "?payment=success"
        ),
        cancel_url=f"{settings.FRONTEND_URL}/analyze?payment=cancelled",
        metadata=metadata,
        payment_intent_data={
            "metadata": metadata,  # Also attach to PaymentIntent for refund lookups
        },
        expires_at=None,  # Default 24-hour expiry
    )
    return session
```

9.2 Complete Payment Flow Sequence

```
┌──────────┐        ┌──────────┐        ┌──────────┐        ┌──────────────┐
│ Browser  │        │ FastAPI  │        │ Stripe   │        │ Celery Workers │
└────┬─────┘        └────┬─────┘        └────┬─────┘        └──────┬───────┘
     │                   │                   │                     │
     │ POST /analyze     │                   │                     │
     │ {address, email...} ──►               │                     │
     │                   │ geocode(address)  │                     │
     │                   │ INSERT analyses   │                     │
     │                   │ INSERT agent_results                    │
     │                   │ (7 × pending)     │                     │
     │                   │                   │                     │
     │                   │ Session.create() ─────►                 │
     │                   │◄── {session.id,   │                     │
     │                   │    checkout_url}  │                     │
     │                   │                   │                     │
     │◄── {analysis_id,  │                   │                     │
     │     checkout_url} │                   │                     │
     │                   │                   │                     │
     │ window.location.href = checkout_url   │                     │
     │──────────────────────────────────────►│                     │
     │                   │                   │                     │
     │◄── Stripe Checkout page (hosted)      │                     │
     │ User enters card + submits            │                     │
     │                   │                   │                     │
     │                   │  POST /webhook/stripe                   │
     │                   │◄──────────────────────                  │
     │                   │ verify signature  │                     │
     │                   │ UPDATE analyses   │                     │
     │                   │   payment_status=paid                   │
     │                   │   status=processing                     │
     │                   │                   │                     │
```

```
      |                    | dispatch chord()      |                    |
      |                    | ──────────────────────────────────────────► |
      |                    |                       |                    | 7 agents r
      |                    |◄── 200 OK ────────────|                    | in parall
      |                    |                       |                    |
      | Stripe redirects to /status/{analysis_id}  |                    |
      |◄──────────────────────────────────────────| |                    |
      |                    |                       |                    |
      | GET /api/v1/analyses/{id} (polling)        |                    |
      |────────────────────►|                      |                    |
      |◄── {status, agent_results[]}               |                    |
      | (repeat every 5s)  |                       | agents complete    |
      |                    |                       |◄──────────────────|
      |                    |                       | aggregate task runs |
      |                    |                       | PDF generated      |
      |                    |                       | S3 upload          |
      |                    |                       | email sent         |
      |                    |                       | status=complete    |
      | GET /api/v1/analyses/{id} → status=complete |                    |
      |────────────────────►|                      |                    |
      |◄── {status: complete, overall_score: 74}   |                    |
      | router.push(/report/{id})                  |                    |
```

9.3 Webhook Security

The Stripe webhook endpoint verifies signatures using `stripe.Webhook.construct_event`. This uses the `STRIPE_WEBHOOK_SECRET` (a `whsec_...` value from the Stripe Dashboard). The raw request body must be passed without any JSON parsing — FastAPI's `await request.body()` provides this.

Idempotency: Stripe may deliver the same webhook event multiple times. The endpoint is idempotent — if `payment_status` is already `paid`, the analysis dispatch is skipped using:

```
if analysis.payment_status == PaymentStatus.PAID:
    return JSONResponse({"received": True, "detail": "already_processed"})
```

10. Environment Variables

```
# ── .env.example ──────────────────────────────────────────────────────────
# Copy to .env and fill in all values before running.
# Never commit .env to source control.

# ── Database ──────────────────────────────────────────────────────────────
# Railway PostgreSQL connection string (Railway provides this automatically)
DATABASE_URL=postgresql://skybase_user:CHANGE_ME@localhost:5432/skybase_dev

# ── Redis / Celery ────────────────────────────────────────────────────────
# Redis for Celery broker + result backend
# Local: redis://localhost:6379/0
# Upstash: rediss://username:password@endpoint:port
REDIS_URL=redis://localhost:6379/0
```

```
# ── Stripe ──────────────────────────────────────────────────────────────
# Live keys: sk_live_... / pk_live_...
# Test keys: sk_test_... / pk_test_...
STRIPE_SECRET_KEY=sk_test_REPLACE_ME
STRIPE_PUBLISHABLE_KEY=pk_test_REPLACE_ME
# whsec_... from Stripe Dashboard → Webhooks → your endpoint → Signing secret
STRIPE_WEBHOOK_SECRET=whsec_REPLACE_ME
# price_... — the $499 product price ID from Stripe Dashboard
STRIPE_PRICE_ID=price_REPLACE_ME

# ── Google Maps ─────────────────────────────────────────────────────────
# Required for: address autocomplete (frontend), geocoding fallback (backend)
# Enable: Maps JavaScript API, Places API, Geocoding API
GOOGLE_MAPS_API_KEY=REPLACE_ME

# ── EIA API ─────────────────────────────────────────────────────────────
# Free API key from: https://www.eia.gov/opendata/register.php
EIA_API_KEY=REPLACE_ME

# ── NREL ────────────────────────────────────────────────────────────────
# Free API key from: https://developer.nrel.gov/signup/
NREL_API_KEY=REPLACE_ME

# ── AWS S3 / Cloudflare R2 ──────────────────────────────────────────────
# For Cloudflare R2: set AWS_S3_ENDPOINT_URL to your R2 account endpoint
AWS_S3_BUCKET=skybase-reports
AWS_ACCESS_KEY_ID=REPLACE_ME
AWS_SECRET_ACCESS_KEY=REPLACE_ME
AWS_REGION=us-east-1
# Cloudflare R2 endpoint (leave blank for AWS S3):
# AWS_S3_ENDPOINT_URL=https://ACCOUNT_ID.r2.cloudflarestorage.com

# ── Email (SendGrid) ────────────────────────────────────────────────────
# Create API key at: https://app.sendgrid.com/settings/api_keys
# Permission: Mail Send
SENDGRID_API_KEY=SG.REPLACE_ME
FROM_EMAIL=reports@skybaseintel.com
FROM_NAME=SkyBase Intelligence

# ── Application ─────────────────────────────────────────────────────────
ENVIRONMENT=development  # development | staging | production
SECRET_KEY=REPLACE_WITH_32_RANDOM_BYTES  # Used for signed tokens if needed

# ── Frontend (Next.js) ──────────────────────────────────────────────────
# These must be prefixed NEXT_PUBLIC_ to be available in the browser
NEXT_PUBLIC_API_URL=http://localhost:8000
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_test_REPLACE_ME
NEXT_PUBLIC_GOOGLE_MAPS_API_KEY=REPLACE_ME

# ── CORS ────────────────────────────────────────────────────────────────
# Comma-separated list of allowed frontend origins
CORS_ORIGINS=["http://localhost:3000","https://skybaseintel.com"]

# ── Frontend URL ────────────────────────────────────────────────────────
# Used by backend for Stripe redirect URLs
```

```
FRONTEND_URL=http://localhost:3000

# ── OpenAIP (FAA airspace data) ──────────────────────────────────────────
# Free tier at: https://www.openaip.net/
OPENAIP_API_KEY=REPLACE_ME
```

11. Deployment Architecture

11.1 Infrastructure Overview

```
GitHub Repository (monorepo)
        │
        ├── Push to main
        │        │
        │    GitHub Actions
        │    ├ Lint (ruff, eslint)
        │    ├ Tests (pytest, jest)
        │    └ Build check
        │        │
        │    ┌────┴────┐
        │    │         │
        │  Vercel   Railway
        │ (frontend) (backend)
        │    │         │
        │  CDN    Docker containers
        │ (global)  ├ FastAPI web process
        │           ├ Celery agent workers (×2-4)
        │           ├ Celery aggregation worker (×1)
        │           ├ PostgreSQL (Railway managed)
        │           └ Redis (Railway managed)
        │
        └── External Services
             ├ AWS S3 / Cloudflare R2 (PDF storage)
             ├ Stripe (payments)
             ├ SendGrid (email delivery)
             └ Sentry (error tracking)
```

11.2 Railway Configuration

Railway auto-deploys from the `backend/` directory. The root `railway.json` configures separate services:

```
{
  "$schema": "https://railway.app/railway.schema.json",
  "build": {
    "builder": "DOCKERFILE",
    "dockerfilePath": "backend/Dockerfile"
  },
  "deploy": {
    "startCommand": "uvicorn app.main:app --host 0.0.0.0 --port $PORT",
    "healthcheckPath": "/health",
```

```
    "healthcheckTimeout": 30,
    "restartPolicyType": "ON_FAILURE",
    "restartPolicyMaxRetries": 3
  }
}
```

A second Railway service uses `backend/Dockerfile.worker` with start command:

```
celery -A app.celery_app worker --loglevel=info -Q agents,aggregation --concurrency=
```

11.3 Backend Dockerfile

```
# backend/Dockerfile
FROM python:3.11-slim

# Install WeasyPrint system dependencies
RUN apt-get update && apt-get install -y --no-install-recommends \
    libpango-1.0-0 \
    libpangoft2-1.0-0 \
    libpangocairo-1.0-0 \
    libcairo2 \
    libgdk-pixbuf2.0-0 \
    libffi-dev \
    shared-mime-info \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

# Run Alembic migrations then start server
CMD ["sh", "-c", "alembic upgrade head && uvicorn app.main:app --host 0.0.0.0 --por
```

```
# backend/Dockerfile.worker
FROM python:3.11-slim

RUN apt-get update && apt-get install -y --no-install-recommends \
    libpango-1.0-0 libpangoft2-1.0-0 libpangocairo-1.0-0 \
    libcairo2 libgdk-pixbuf2.0-0 libffi-dev \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
```

```
CMD ["celery", "-A", "app.celery_app", "worker", \
     "--loglevel=info", "-Q", "agents,aggregation", "--concurrency=4"]
```

11.4 requirements.txt

```
# Web framework
fastapi==0.115.0
uvicorn[standard]==0.32.0
python-multipart==0.0.12

# Database
sqlalchemy==2.0.36
asyncpg==0.30.0
psycopg2-binary==2.9.10
alembic==1.14.0

# Task queue
celery==5.4.0
redis==5.2.0

# PDF generation
weasyprint==62.3
reportlab==4.2.5
jinja2==3.1.4

# HTTP client
httpx==0.28.0

# Payments
stripe==11.2.0

# Email
sendgrid==6.11.0

# Cloud storage
boto3==1.35.0

# Validation + settings
pydantic==2.10.0
pydantic-settings==2.6.0
pydantic[email]==2.10.0

# Utilities
python-dotenv==1.0.1
structlog==24.4.0
sentry-sdk[fastapi]==2.19.0
```

11.5 Vercel Configuration (`frontend/next.config.ts`)

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  output: "standalone",
  images: {
    remotePatterns: [
      { protocol: "https", hostname: "*.r2.cloudflarestorage.com" },
      { protocol: "https", hostname: "*.s3.amazonaws.com" },
    ],
  },
  async rewrites() {
    return [
      {
        source: "/api/:path*",
        destination: `${process.env.NEXT_PUBLIC_API_URL}/api/:path*`,
      },
    ];
  },
};

export default nextConfig;
```

11.6 CI/CD Pipeline

```
# .github/workflows/ci.yml
name: CI

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  backend-test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_USER: test
          POSTGRES_PASSWORD: test
          POSTGRES_DB: skybase_test
        ports: ["5432:5432"]
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
      redis:
        image: redis:7
        ports: ["6379:6379"]
```

```
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
          cache: pip
          cache-dependency-path: backend/requirements.txt

      - name: Install dependencies
        run: pip install -r backend/requirements.txt pytest pytest-asyncio

      - name: Lint (ruff)
        run: |
          pip install ruff
          ruff check backend/app

      - name: Run tests
        working-directory: backend
        env:
          DATABASE_URL: postgresql://test:test@localhost:5432/skybase_test
          REDIS_URL: redis://localhost:6379/0
          STRIPE_SECRET_KEY: sk_test_dummy
          STRIPE_WEBHOOK_SECRET: whsec_dummy
          STRIPE_PRICE_ID: price_dummy
          GOOGLE_MAPS_API_KEY: dummy
          EIA_API_KEY: dummy
          NREL_API_KEY: dummy
          AWS_S3_BUCKET: dummy
          AWS_ACCESS_KEY_ID: dummy
          AWS_SECRET_ACCESS_KEY: dummy
          SENDGRID_API_KEY: dummy
        run: pytest tests/ -v --tb=short

  frontend-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "20"
          cache: npm
          cache-dependency-path: frontend/package.json

      - name: Install dependencies
        working-directory: frontend
        run: npm ci

      - name: Type check
        working-directory: frontend
        run: npx tsc --noEmit

      - name: Lint
        working-directory: frontend
        run: npm run lint
```

```
      - name: Build
        working-directory: frontend
        env:
          NEXT_PUBLIC_API_URL: https://api.skybaseintel.com
          NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY: pk_test_dummy
          NEXT_PUBLIC_GOOGLE_MAPS_API_KEY: dummy
        run: npm run build
```

11.7 Monitoring

- **Sentry**: Add `sentry_sdk.init(dsn=..., integrations=[FastApiIntegration(), CeleryIntegration()])` in `main.py` and `celery_app.py`. Captures unhandled exceptions with full request context.
- **Railway metrics**: CPU, memory, and request throughput visible in the Railway dashboard. Set alerts for memory > 80% (WeasyPrint PDF generation is memory-intensive).
- **Celery monitoring**: Deploy Flower (`celery flower`) as a Railway service for real-time task queue visibility. Restrict access with HTTP Basic Auth.
- **Structured logging**: All log lines output as JSON via `structlog`, shipped to Railway's log drain or a log aggregator (Papertrail, Datadog).

12. Local Development Setup

12.1 docker-compose.yml

```
# docker-compose.yml (repository root)
version: "3.9"

services:
  # ── PostgreSQL ──────────────────────────────────────────────────────────
  postgres:
    image: postgres:16-alpine
    container_name: skybase_postgres
    environment:
      POSTGRES_USER: skybase
      POSTGRES_PASSWORD: skybase_dev_password
      POSTGRES_DB: skybase_dev
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U skybase -d skybase_dev"]
      interval: 5s
      timeout: 5s
      retries: 5

  # ── Redis ───────────────────────────────────────────────────────────────
  redis:
    image: redis:7-alpine
```

```
    container_name: skybase_redis
    ports:
      - "6379:6379"
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5

  # ── FastAPI Backend ──────────────────────────────────────────────────────
  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: skybase_backend
    ports:
      - "8000:8000"
    volumes:
      - ./backend:/app  # Hot reload in development
    env_file:
      - .env
    environment:
      DATABASE_URL: postgresql://skybase:skybase_dev_password@postgres:5432/skybase_
      REDIS_URL: redis://redis:6379/0
      ENVIRONMENT: development
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    command: >
      sh -c "alembic upgrade head &&
             uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload"

  # ── Celery Agent Workers ─────────────────────────────────────────────────
  celery_worker:
    build:
      context: ./backend
      dockerfile: Dockerfile.worker
    container_name: skybase_celery_agents
    volumes:
      - ./backend:/app
    env_file:
      - .env
    environment:
      DATABASE_URL: postgresql://skybase:skybase_dev_password@postgres:5432/skybase_
      REDIS_URL: redis://redis:6379/0
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    command: >
      celery -A app.celery_app worker
             --loglevel=info
             -Q agents,aggregation
```

```
            --concurrency=4

  # ── Celery Flower (task monitoring UI) ──────────────────────────────────
  flower:
    image: mher/flower:2.0
    container_name: skybase_flower
    command: celery flower --broker=redis://redis:6379/0 --port=5555
    ports:
      - "5555:5555"
    depends_on:
      - redis

  # ── Next.js Frontend ────────────────────────────────────────────────────
  frontend:
    image: node:20-alpine
    container_name: skybase_frontend
    working_dir: /app
    volumes:
      - ./frontend:/app
      - /app/node_modules   # Anonymous volume prevents host node_modules override
      - /app/.next
    ports:
      - "3000:3000"
    environment:
      NEXT_PUBLIC_API_URL: http://localhost:8000
      NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY: ${NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY}
      NEXT_PUBLIC_GOOGLE_MAPS_API_KEY: ${NEXT_PUBLIC_GOOGLE_MAPS_API_KEY}
    command: sh -c "npm install && npm run dev"

volumes:
  postgres_data:
```

12.2 Step-by-Step Setup

Prerequisites:

- Docker Desktop ≥ 4.25
- Node.js 20+ (for IDE tooling only; runtime is containerized)
- Python 3.11+ (for IDE tooling only)

Setup:

```
# 1. Clone the repository
git clone https://github.com/your-org/skybase.git
cd skybase

# 2. Copy and configure environment variables
cp .env.example .env
# Edit .env:
#   - Set STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET, STRIPE_PRICE_ID
#   - Set GOOGLE_MAPS_API_KEY
#   - Set EIA_API_KEY (free at https://www.eia.gov/opendata/register.php)
#   - Set NREL_API_KEY (free at https://developer.nrel.gov/signup/)
#   - Set AWS credentials OR Cloudflare R2 credentials
```

```
#   - Set SENDGRID_API_KEY
#   - Leave DATABASE_URL and REDIS_URL as-is (docker-compose overrides them)

# 3. Start all services
docker compose up --build

# 4. Verify services are running
# Backend API: http://localhost:8000/docs
# Frontend:    http://localhost:3000
# Flower:      http://localhost:5555

# 5. Run database migrations (auto-runs on backend start, but explicit command avail
docker compose exec backend alembic upgrade head

# 6. (Optional) Stripe webhook forwarding for local development
# Install Stripe CLI: https://stripe.com/docs/stripe-cli
stripe listen --forward-to localhost:8000/api/v1/analyses/webhook/stripe
# This prints a whsec_... value — set it as STRIPE_WEBHOOK_SECRET in .env
# Then restart the backend: docker compose restart backend

# 7. Run backend tests
docker compose exec backend pytest tests/ -v

# 8. Run frontend type checking
docker compose exec frontend npx tsc --noEmit
```

Useful development commands:

```
# Tail backend logs
docker compose logs -f backend

# Tail Celery worker logs
docker compose logs -f celery_worker

# Create a new Alembic migration after model changes
docker compose exec backend alembic revision --autogenerate -m "describe your chang

# Access PostgreSQL directly
docker compose exec postgres psql -U skybase -d skybase_dev

# Manually trigger an analysis task (for testing, bypassing payment)
docker compose exec backend python -c "
from app.tasks.orchestrator import dispatch_analysis
dispatch_analysis(
    analysis_id='00000000-0000-0000-0000-000000000001',
    lat=25.7617,
    lon=-80.1918,
    address='1234 NW 56th St, Miami, FL 33142',
    property_context={'property_type': 'rooftop', 'build_year': 1998, 'num_stories':
)
"

# Reset the local database
docker compose down -v && docker compose up --build
```

13. Development Timeline Estimate

Component	Estimated Build Time	Notes
Database schema + migrations	2 hours	SQLAlchemy models, Alembic migration, verify with `alembic upgrade head`
FastAPI boilerplate + health	1 hour	App factory, CORS, deps, health endpoint, Sentry integration
Stripe payment flow	3 hours	Checkout session creation, webhook handler, signature verification, idempotency
Geocoding service	2 hours	Nominatim primary, Google Maps fallback, error handling
Agent 1 (Airspace)	4 hours	OpenAIP API integration, airport proximity scoring, FAA class mapping
Agent 2 (Zoning)	6 hours	OSM Overpass API, state GIS portal integrations, tag-to-zoning classification
Agent 3 (Power)	3 hours	NREL Utility Rate API, OSM substation lookup, scoring model
Agent 4 (Structural)	2 hours	Rules-based engine, FAA EB 105A criteria, no external API
Agent 5 (Regulatory)	3 hours	Static state ruleset DB, FDOT/CEQA/SEQRA special cases
Agent 6 (Cost)	2 hours	Internal cost model, ROI calculator, ranges by property type
Agent 7 (Noise/Environment)	3 hours	NOAA METAR wind API, FEMA flood zone API, OSM noise-sensitive uses
Celery orchestration	3 hours	Group + chord setup, partial failure handling, DB status updates
PDF report template	6 hours	WeasyPrint HTML/CSS template, Jinja2 rendering, ReportLab fallback
S3 upload + signed URLs	1 hour	boto3 upload, presigned URL generation with expiry
Email delivery (SendGrid)	1 hour	Report delivery email with download link
Next.js frontend	8 hours	4 pages, 6 components, TypeScript types, API client
Google Places autocomplete	1 hour	Script loader, Places Autocomplete widget integration
Deployment pipeline	2 hours	Dockerfiles, Railway config, Vercel config, GitHub Actions CI
End-to-end testing	3 hours	Full payment → agents → PDF → email flow with test Stripe keys
Total	~56 hours	Achievable in 6-7 focused days solo, or 3-4 days with two engineers

Recommended build order:

1. Database schema → FastAPI boilerplate → health endpoint (Day 1)
2. Stripe payment flow → geocoding → Celery infrastructure (Day 2)

3. Agents 4, 5, 6 (rules-based, no external API) → Agents 3, 7 (clean APIs) (Day 3)
4. Agents 1, 2 (complex external data) → orchestration layer (Day 4)
5. PDF template → S3 + email → end-to-end test (Day 5)
6. Next.js frontend → deployment pipeline → production smoke test (Day 6)

Risk items:

- **Zoning (Agent 2)**: State GIS portals vary in API quality and rate limits. OSM fallback is always available but less precise. Budget extra time.
- **WeasyPrint system deps**: Cairo/Pango libraries can be difficult on non-Debian environments. Docker eliminates this completely — do not test outside Docker.
- **Stripe webhook local testing**: Stripe CLI is required. Document the `stripe listen` command in the README prominently.
- **FAA DOF bulk download**: The FAA DOF is a large CSV (~100MB). Consider pre-processing it into a PostGIS table rather than parsing it on every agent run. This is a Day 1 infrastructure decision.

End of SkyBase Intelligence Platform Technical Architecture Document
Document version 1.0.0 — SkyBase Engineering